

University of Waterloo E-Thesis Template for L^AT_EX

by

Pat Neugraad

A thesis
presented to the University of Waterloo
in fulfillment of the
thesis requirement for the degree of
Doctor of Philosophy
in
Philosophy of Zoology

Waterloo, Ontario, Canada, 2023

© Pat Neugraad 2023

Examining Committee Membership

The following served on the Examining Committee for this thesis. The decision of the Examining Committee is by majority vote.

External Examiner: Bruce Bruce
Professor, Dept. of Philosophy of Zoology, University of Wallamaloo

Supervisor(s): Ann Elk
Professor, Dept. of Zoology, University of Waterloo
Andrea Anaconda
Professor Emeritus, Dept. of Zoology, University of Waterloo

Internal Member: Pamela Python
Professor, Dept. of Zoology, University of Waterloo

Internal-External Member: Meta Meta
Professor, Dept. of Philosophy, University of Waterloo

Other Member(s): Leeping Fang
Professor, Dept. of Fine Art, University of Waterloo

Author's Declaration

I hereby declare that I am the sole author of this thesis. This is a true copy of the thesis, including any required final revisions, as accepted by my examiners.

I understand that my thesis may be made electronically available to the public.

Abstract

This is the abstract.

Acknowledgements

I would like to thank all the little people who made this thesis possible.

Dedication

This is dedicated to the one I love.

Table of Contents

Examining Committee	ii
Author's Declaration	iii
Abstract	iv
Acknowledgements	v
Dedication	vi
List of Figures	ix
List of Tables	x
1 Introduction	1
1.1 An overview of TLS 1.3	1
1.1.1 Certificate-based authentication	2
1.1.2 Key schedule and HKDF	5
1.1.3 Security analysis	5
1.1.4 Transition to post-quantum	5
1.2 Post-quantum IoT security	7
1.2.1 Software optimization	7
1.2.2 Hardware accelerations	7
1.2.3 Authenticating the embedded device	8

2	Optimizing post-quantum TLS	9
2.1	KEMTLS	9
2.1.1	KEMTLS Handshake	9
2.1.2	KEMTLS security	12
2.1.3	KEMTLS performance	13
2.2	IND-1CCA KEM	13
2.2.1	The encrypt-then-MAC transformation	14
2.2.2	Related works	20
3	Implementation	22
3.1	WolfSSL	22
3.1.1	Integrating additional post-quantum algorithms	22
3.1.2	Implementing KEMTLS	23
3.1.3	Miscellaneous comments	26
3.2	Hardware setup	26
4	Performance of post-quantum TLS 1.3 and KEMTLS	28
4.1	Communication cost	28
4.2	Cryptographic operations performance	31
4.3	Handshake latency	31
4.3.1	Discussion	33
5	Conclusion and future works	38
	References	39
	APPENDICES	50

List of Figures

1.1	TLS 1.3 handshake with unilateral authentication	3
1.2	Certificate chain and PKI	4
1.3	HKDF in TLS 1.3 key schedule	4
2.1	KEMTLS handshake with unilateral authentication	11
2.2	The encrypt-then-MAC KEM routines	15
2.3	Sequence of games in the proof of Theorem 2.2.1	17
2.4	OW-PCA adversary B simulates game 3 for IND-CCA adversary A in the proof for Theorem 2.2.1	18
2.5	T_H transformation	21
4.1	Key agreement performance (microseconds).	35
4.2	Authentication durations (microseconds).	36
4.3	Handshake durations (microseconds).	37

List of Tables

4.1	Key agreement communication costs (bytes).	29
4.2	Digital signatures communication costs (bytes)	29
4.3	Server certificate chain size (bytes). Each certificate is encoded in PEM format. For rows with one name, there are still three certificates, but they use the same scheme for all three keys.	30
4.4	Key exchange performance on RP2350	31
4.5	Digital signature performance on RP2350	32
4.6	Comparing key agreement performance (microseconds).	34
4.7	Medium authentication durations (microseconds).	34
4.8	Medium handshake durations (microseconds).	35

Chapter 1

Introduction

1.1 An overview of TLS 1.3

Transport Layer Security (TLS) is a communication protocol with which two parties can exchange data in the presence of passive or active adversaries while preserving **confidentiality, integrity, and authenticity**. It was first developed at Netscape in 1994 and went through many iterations. The latest revision is TLS 1.3, formally standardized by IETF in RFC 8446 [65] in 2018. TLS 1.3 made significant improvements over TLS 1.2 [26] and prior versions, including deprecating weak symmetric cipher suites, requiring forward secrecy at all times, and restructuring the handshake state machine in order to simplify the handshake workflow. According to Cloudflare [86], TLS 1.3 reached more than 93% adoption on the Internet. Given TLS 1.3’s superior design and nearly universal adoption over other versions of TLS, this work will focus on this version unless otherwise specified.

Figure 1.1 illustrates a simple TLS 1.3 handshake over TCP. A new TLS 1.3 connection begins with a handshake in which the two parties exchange a specific sequence of messages to negotiate cryptographic parameters, establish shared secrets, and authenticate each other. If the handshake is successful, then the connection transitions to exchanging application data using the encryption scheme and secret keys obtained from the handshake phase. In TLS 1.3, the handshake can be further divided into two phases:

1. Key exchange

- (a) **ClientHello**: client publishes its supported symmetric cipher suites (AEAD, hash functions), signature algorithms, a random nonce, and key exchange public keys.

- (b) **ServerHello**: server chooses its preferred symmetric cipher suite, as well as publishes its own random nonce and key exchange public keys.

2. Authentication

- (a) **Certificates**: server sends a chain of X.509 certificates that bind its identity to a digital signature public key; optionally, server can request client authentication (**CertificateRequest**), in which case the client must also send a chain of certificates.
- (b) **CertificatesVerify**: server proves ownership of the corresponding digital signature secret key by producing a signature over the handshake transcript; client will do the same upon server's request
- (c) **Finished**: both parties prove to each other the integrity of the handshake by producing an HMAC over the handshake transcript

1.1.1 Certificate-based authentication

Authentication in TLS 1.3 relies on digital signatures and X.509 certificates. Figure [1.2](#) illustrates a certificate chain.

Each X.509 certificate encodes three pieces of information: identities of the issuer and the subject, a cryptographic public key, and a cryptographic signature over the identity and the public key. Through this signature, the issuer testifies the subject's ownership of the public key and binds the public key to the identity. The signature of the issuer is verified using the issuer's public key, which is usually presented via another certificate of identical format. This creates a chain of trust in which each certificate is verified by the public key in another certificate, up to the root of trust, which is usually a self-signed certificate whose signature is verified by the public key on the same certificate.

The *Public Key Infrastructure (PKI)* consists of entities that play different roles in this chain of trust. The server (and optionally client) is the leaf node of the chain, and the entities issuing certificates are called *Certificate Authorities (CA)*. The root of this chain is thus referred to as *root CA*, while other issuers between the leaf and the root are referred to as intermediate CAs. A selection of trusted root CA certificates are typically pre-installed in client devices by the vendors (e.g. Google, Mozilla, Apple, Microsoft, etc.).

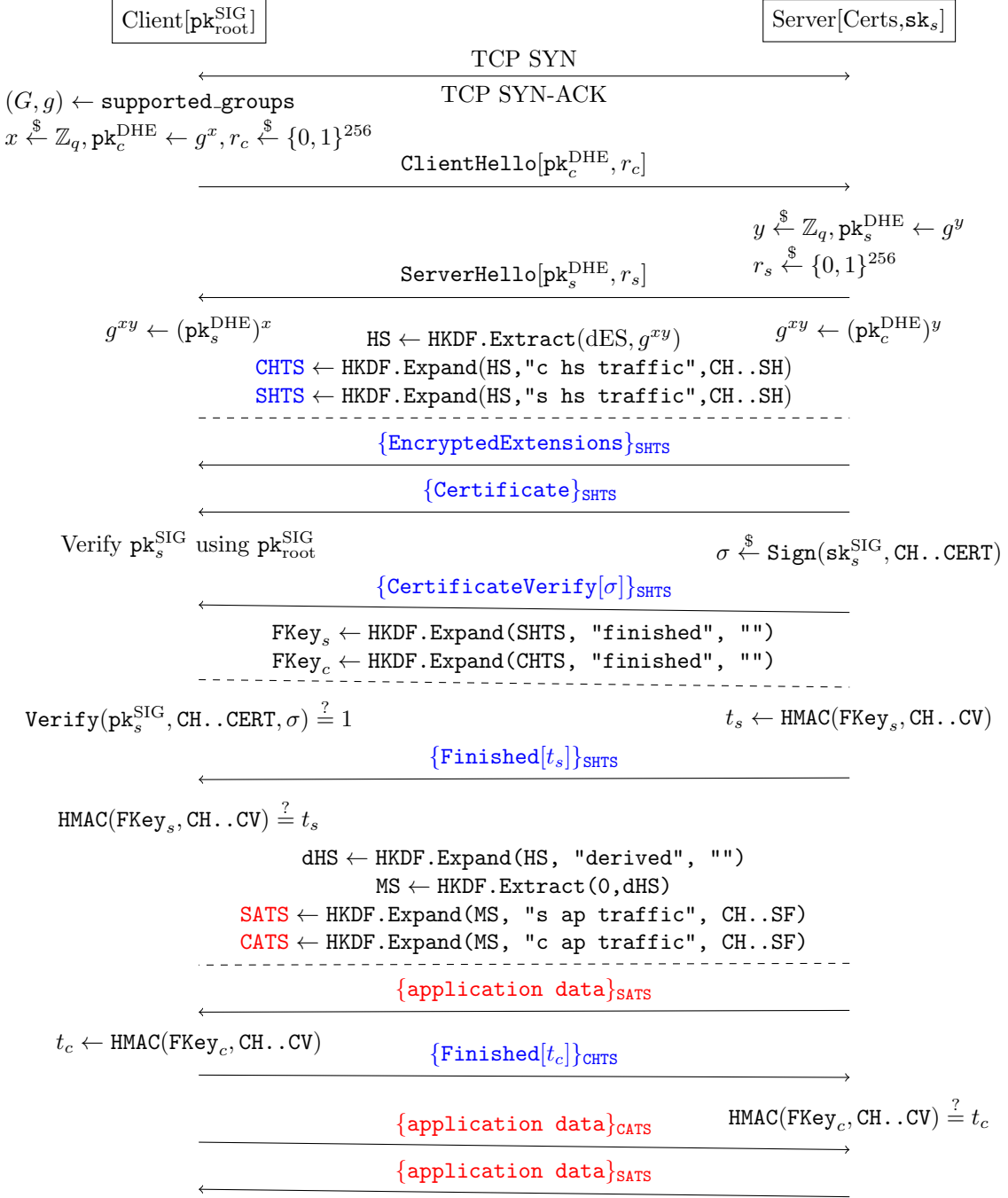


Figure 1.1: TLS 1.3 handshake with unilateral authentication

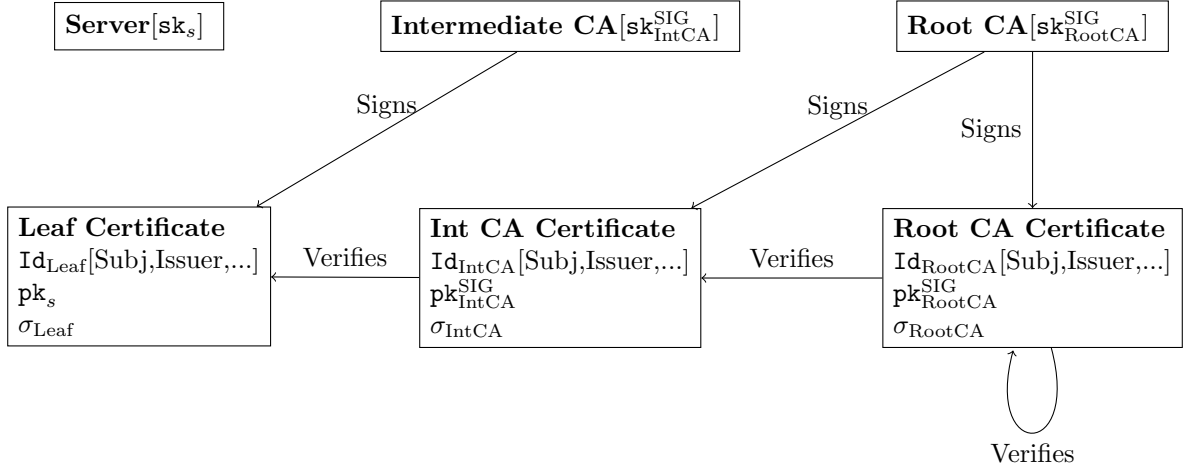


Figure 1.2: Certificate chain and PKI

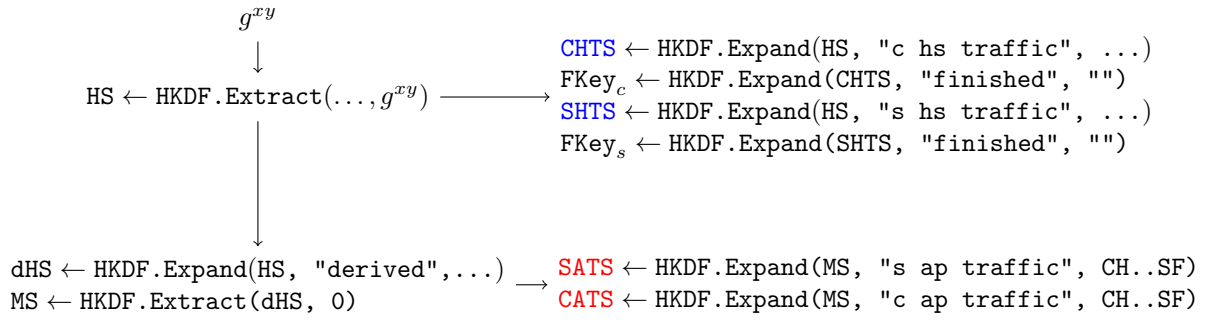


Figure 1.3: HKDF in TLS 1.3 key schedule

1.1.2 Key schedule and HKDF

TLS 1.3 uses the HMAC-based Key Derivation Function (HKDF) [51] to derive cryptographic keys from input keying material (Figure 1.3). Using an extract-and-expand KDF is necessary because the n -bit shared secret obtained from public-key cryptography (e.g. g^{xy} in a Diffie-Hellman key exchange) is not a uniformly random n -bit string [50]. Instead, it typically contains fewer bits of computational entropy.¹

In TLS 1.3, the function `HKDF.Extract` serves as an entropy extractor that concentrates the available randomness in the input keying material. Another critical function of HKDF in TLS 1.3 is to derive computationally independent keys via `HKDF.Expand`. As illustrated in Figure 1.1, all handshake messages following `ServerHello`, as well as all application data, are encrypted using different symmetric keys. Specifically, handshake messages are protected using a pair of handshake traffic keys—client handshake traffic secret (CHTS) and server handshake traffic secret (SHTS)—while application data is encrypted under a separate pair of application traffic keys—client application traffic secret (CATS) and server application traffic secret (SATS). Additionally, there is a unique symmetric key for each party to produce the HMAC tag used in their respective `Finished` messages. According to the HKDF model described in [52, 50], each of these keys—CATS, SATS, CHTS, SHTS, the client finished key, and the server finished key—is computationally independent from all others.

1.1.3 Security analysis

What are the security goals of TLS 1.3? How does TLS achieve them? Read and understand the formal analysis, summarize them here.

1.1.4 Transition to post-quantum

The security of TLS 1.3 was thoroughly analyzed from a theoretical perspective through a long line of work [28, 29, 9, 31, 30]. From a high level, TLS 1.3 aims to achieve confidentiality, integrity, authenticity, forward secrecy, and resistance to replay attacks in the presence of an adversary who controls the network, can passively eavesdrop and/or actively alter the communication, can compromise long-term asymmetric private keys, and can compromise

¹As stated in [50], an adversary with knowledge of g^x and g^y can deterministically compute g^{xy} , implying that the statistical entropy of g^{xy} given g^x and g^y is zero.

some derived session keys², all across multiple sessions. The protocol’s ability to achieve this goal depends on the strength of the various cryptographic primitives: the entropy of the nonces (client and server’s `random` in the `Hello` messages), the collision resistance of the hash function (used for maintaining handshake transcript), the order of the ephemeral Diffie-Hellman group, the EUF-CMA security of the signature, the PRF security of the HKDF, the IND-CCA security of the AEAD, etc.

The advent of large-scale quantum computers affects all cryptographic aspects of the TLS 1.3 protocol. For example, Grover’s quantum search algorithm [40] would half the number of steps required to search through a key space, thereby halving the security of symmetric ciphers. Brassard et al. [17] presented a quantum algorithm that finds hash function collisions in $O(2^{\frac{n}{3}})$ hash queries instead of $O(2^{\frac{n}{2}})$ queries in the classical setting. Fortunately, the asymptotic security of symmetric primitives remains unchanged to this day, so the quantum threat can be straightforwardly mitigated by adopting “bigger” parameters, such as upgrading AES-128 to AES-256, and upgrading SHA256 to SHA512. On the other hand, all asymmetric primitives used in TLS 1.3 (ECDHE, RSA, ECDSA, and EdDSA) are based on hard number-theoretic problems, which Peter Shor’s quantum algorithm [72, 73] can solve with polynomial complexity. Therefore, using bigger parameters is not sufficient for making the public-key primitives of TLS 1.3 quantum resistant; instead, entirely new classes of cryptographic schemes for which no efficient quantum algorithm is known are required. Due to the “harvest-now-decrypt-later” attacks, adopting quantum-resistant key exchange algorithms takes the highest urgency, although due to the need for proven security in real-world usage, the transition to entirely post-quantum cryptography must start as soon as possible.

At the time of writing this thesis, the NIST post-quantum cryptography standardization project has standardized one key encapsulation mechanism based on hard lattice problems (ML-KEM) and two digital signature algorithms, one based on lattice problems (ML-DSA) and one based on hash functions (SLH-DSA). There are two more algorithms (Falcon and HQC) that are currently going through the standardization process.

²The security model does not permit the adversary to directly compromise certain critical secrets and randomness, such as the Diffie-Hellman shared secret, since TLS 1.3 is not designed to be secure if these secrets are compromised

1.2 Post-quantum IoT security

1.2.1 Software optimization

pqm4 [48].

1.2.2 Hardware accelerations

Because embedded devices have limited computing capacity compared to desktop processors, hardware acceleration is a popular strategy for implementing cryptographic protocols on embedded devices. In this section we briefly describe some popular strategies for accelerating cryptographic operations in constrained environments.

Accelerating symmetric operations. Hardware acceleration for symmetric primitives has been widely available to modern processors. Dedicated AES instructions were first brought to Intel processors with the Westmere microarchitecture in 2010³. On embedded devices, ARMv8’s Cryptography Extensions included hardware-accelerated AES/SHA256 instructions in 2011⁴. The widespread availability of hardware-accelerated symmetric primitives motivated many post-quantum cryptographic schemes to include variants that use AES/SHA256 as alternative to Keccak for implementing pseudorandom functions (PRF) and pseudorandom generators (PRG). For example, Kyber’s round 2 submission included a “90s-variant” that used AES in counter mode and SHA2 instead of Keccak, and optimized implementation of the 90s-variant offered significant performance improvements over the optimized implementation of “standard” variant using Keccak [?]. Similarly, hash-based signature SPHINCS+ also offered variants that used SHA2 instead of Keccak [?].

Accelerating modular arithmetics. A second strategy for accelerating post-quantum cryptography is to target the modular arithmetics. Besides hardware accelerations dedicated to specific PQC scheme [44, 49, 4], there are also active efforts at re-purposing RSA/ECC hardware accelerators [1]. Banerjee et al. [7] combined hardware acceleration for both symmetric primitives and modular arithmetics and achieved order-of-magnitude speed and energy efficiency improvements over software-only implementations.

³Citation needed

⁴<http://infocenter.arm.com/help/index.jsp?topic=/com.arm.doc.ddi0500e/CJHDEBAF.html>

1.2.3 Authenticating the embedded device

Prior works [83, 19, 39] evaluating post-quantum TLS on embedded systems primarily focused on configurations in which the constrained device is an anonymous TLS client who does not need to authenticate itself to the server. While the TLS 1.3 (and KEMTLS) supports mutual authentication, implementing it on an embedded device poses unique challenges. The same set of challenges also apply to using embedded device as TLS server.

The primary obstacle to implementing authentication on embedded device is the threat of physical intrusion. For example, servers in a data center are physically secured within a building guarded by human with guns, so they often do not require extra hardware security measures. In some critical use cases where extra security is required (e.g. banking), a hardware security module (HSM) can be deployed on premise. By comparison, embedded devices are commonly deployed in unprotected if not hostile environments, where adversaries can easily capture the device and execute sophisticated physical attacks.

There are numerous hardware and software measures that can be deployed to protect secrets on an embedded devices, including but not limited to physical unclonable functions (PUF), trusted platform modules (TPM), cryptocoprocessors, and secure boot. In addition to cost and power consumption concerns, however, the existence of a “secure storage” means one can simply deploy a 256-bit symmetric key and use symmetric primitives (i.e. HMAC) to authenticate the device. In fact, TLS 1.3 supports handshake using pre-shared key (PSK)[65] in environments with no public key infrastructure (PKI). PSK automatically provides mutual authentication and also offers vastly superior performance by eliminating all asymmetric cryptographic operations. In other words, regardless of whether private keys can be securely stored or not, there is no scenario in which authenticating an embedded device using public-key cryptography makes sense.

Chapter 2

Optimizing post-quantum TLS

This section, we introduce two optimization on post-quantum TLS: Section [2.1](#) introduces KEMTLS, and Section [2.2](#) introduces IND-1CCA KEM for ephemeral key exchange.

2.1 KEMTLS

KEMTLS [\[69\]](#), first proposed by Schwabe, Stebila, and Wiggers in 2020, is a variation on TLS 1.3 that uses key encapsulation mechanism (KEM) instead of digital signature for peer authentication. KEMTLS is motivated by the observation that post-quantum signature candidates generally have larger public key and signature than post-quantum KEM candidates' public key and ciphertext, so by using KEMs instead of signatures, one can obtain significant savings in communication sizes.

Section [2.1.1](#) will introduce the KEMTLS handshake workflow. Section [2.1.2](#) will review the security properties of KEMTLS and compare them with TLS 1.3. Finally, Section [2.1.3](#) will review the performance benchmarks from existing literature and discuss further optimizations in specific use cases.

2.1.1 KEMTLS Handshake

Figure [2.1](#) illustrates a KEMTLS handshake with unilateral authentication. Like TLS 1.3's handshake (Figure [1.1](#)), KEMTLS handshake can be cleanly divided into a key exchange phases where the two parties establish an ephemeral shared secret, and an authentication

phase where the server proves its identity to the client. However, KEMTLS' authentication flow diverges significantly from TLS 1.3's authentication flow, and there are subtle downstream differences. We will discuss them in detail in this section.

Ephemeral key exchange. Like TLS 1.3, a KEMTLS handshake begins with establishing a TCP stream and client/server exchanging `ClientHello` and `ServerHello` in this order. Client first picks a KEM scheme KEM_e for ephemeral key exchange, then generates the ephemeral keypair $(\text{pk}_e, \text{sk}_e) \xleftarrow{\$} \text{KEM}_e.\text{KeyGen}()$. The ephemeral public key pk_e is sent to the server in `ClientHello`, which also includes a 32-byte client nonce $r_c \in \{0, 1\}^{256}$, client's supported cipher suites, and other parameters not crucial to the workflow of KEMTLS. If server accepts the parameters in `ClientHello`, then it encapsulates a random secret $(\text{ct}_e, \text{ss}_e) \xleftarrow{\$} \text{KEM}_e.\text{Encap}(\text{pk}_e)$. The ciphertext ct_e is included in `ServerHello` sent to the client, alongside server's random nonce $r_s \in \{0, 1\}^{256}$ and other parameters. Finally, after receiving `ServerHello`, client recovers the ephemeral shared secret by decapsulating the ciphertext $\text{ss}_e \leftarrow \text{KEM}_e.\text{Decap}(\text{sk}_e, \text{ct}_e)$.

At this stage, client and server have established an ephemeral shared secret ss_e and will update their key schedule synchronously. Specifically, each extracts a handshake secret (HS) from ss_e using `HKDF.Extract`, then derive client handshake traffic secret (`CHTS`), server handshake traffic secret (`SHTS`), and derived handshake secret (dHS) using `HKDF.Expand` under different labels. CHTS and SHTS will be used to encrypt subsequent handshake messages from the client and the server respectively. dHS on the other hand serves as the secret state of the key schedule and will participate in subsequent key schedule update. Figure 2.1 used color-coding and subscripts to clarify which handshake message is encrypted and under which secret.

(Implicit) Server authentication. Like TLS 1.3, the authentication phase begins with server sending `Certificate`, but the workflow diverges afterwards. With KEMTLS, server's long-term public key pk_s is a KEM encapsulation key. To prove server's possession of the corresponding decapsulation key, client uses server's long-term public key to encapsulate a secret $(\text{ct}_s, \text{ss}_s) \xleftarrow{\$} \text{KEM}_s.\text{Encap}(\text{pk}_s)$, then sends the ciphertext in a handshake message `KemCiphertext`[ct_s]. After receiving `KemCiphertext`[ct_s], server recovers the authentication secret using its long-term secret key $\text{ss}_s \leftarrow \text{KEM}_s.\text{Decap}(\text{sk}_s, \text{ct}_s)$.

After this round of messages, the two parties have obtained another shared secret ss_s and will perform a second update to the key schedule. First, the (unauthenticated) handshake secret (HS) will be upgraded to an authenticated handshake secret (AHS) by

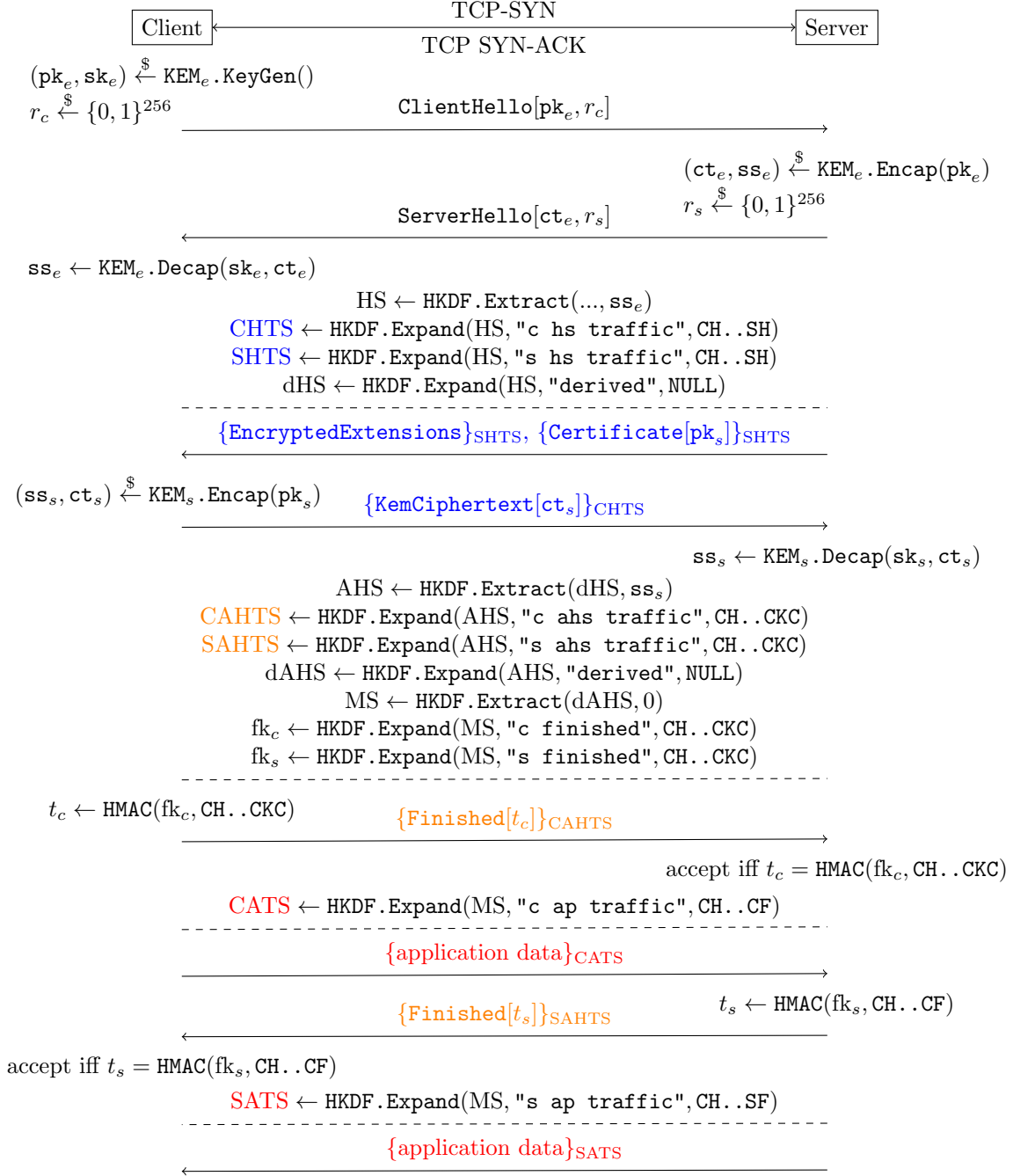


Figure 2.1: KEMTLS handshake with unilateral authentication

extracting randomness from dHS and $\mathbf{ss}_s$. From AHS, the handshake traffic secrets will be upgraded to authenticated handshake traffic secrets (**CAHTS**, **SAHTS**). The key schedule’s secret state is updated from AHS, then used to produce the Master Secret (MS). Last but not least, the **Finished** keys $\mathbf{fk}_c, \mathbf{fk}_s$ are derived from MS.

Key confirmation/explicit authentication. The final round of handshake message includes client and server exchanging **Finished**. Like TLS 1.3, the **Finished** message contains an HMAC tag over the handshake transcript up to the point when the message is sent. Also like TLS 1.3, each party can derive the appropriate application traffic keys (**CATS**, **SATS**) and start sending application data right after sending its own **Finished**. On the other hand, unlike TLS 1.3, the order of “who first sends **Finished** and application” is flipped: where in TLS 1.3, server sends **Finished** before client, in KEMTLS, client sends **Finished** before server. This means that from client’s perspective, TLS 1.3 and KEMTLS require the same one round trip (1-RTT) of handshake messages before client can start sending application data. From server’s perspective, the number of round trips required before sending application data increased, but as [69] pointed out, in most TLS applications, client sends the first round of application data, so the increase of round trips for server is likely irrelevant to the latency of the application. The flipped order of **Finished** also has security implications, since client will be sending its first round of application data before having received server’s **Finished**. We will discuss them in details in Section 2.1.2.

2.1.2 KEMTLS security

KEMTLS has identical security goals to TLS 1.3. Analysis of KEMTLS’ security thus follows the same framework as the formal analysis of TLS 1.3’s security described in Section 1.1.3. A “pen-and-paper” proof of KEMTLS security properties is first presented in [69]. Later, a computer-verified symbolic analysis of KEMTLS was conducted using Tamarin [22, 58]. At a high level, KEMTLS achieves confidentiality, integrity, authenticity, forward secrecy, and resistance to replays if the client/server nonces are sufficiently large, the ephemeral key exchange KEM is IND-1CCA secure, the authentication KEM is IND-CCA secure, the chosen hash function is collision resistant, the HKDF is a secure PRF, and the AEAD is IND-CCA secure. These properties are formally stated by Theorem 2.1.1:

Theorem 2.1.1 (Multi-stage security of KEMTLS [69]). *Let n be the number of sessions and U be the number of parties. The advantage of any probabilistic polynomial-time ad-*

versary $\mathcal{A}$ in breaking the multi-stage security of KEMTLS is bounded by:

$$\begin{aligned} Adv(\mathcal{A}) \leq \frac{n^2}{2^{|r_c|}} + \epsilon_H^{COL} + 6n \bigg(& n(\epsilon_{KEM_e}^{IND-1CCA} + \epsilon_{HKDF.Ext}^{PRF} + 2\epsilon_{HKDF.Ext}^{dual-PRF} + 3\epsilon_{HKDF.Exp}^{PRF} + \epsilon_{HMAC}^{EUF-CMA}) \\ & + U(\epsilon_{KEM_s}^{IND-CCA} + 2\epsilon_{HKDF.Ext}^{dual-PRF} + 2\epsilon_{HKDF.Exp}^{PRF} + \epsilon_{HMAC}^{EUF-CMA}) \bigg) \end{aligned}$$

Implicit authentication. In TLS 1.3, client will not receive server’s application data nor send its own application data until after it has received and validated server’s **Finished**, which means that all application data exchanged are explicitly authenticated. In KEMTLS, however, client can start sending application data after sending its **Finished** but before receiving server’s **Finished**. The application data sent before receiving server’s **Finished** are encrypted under client’s application traffic key, which contains entropy from the authentication secret $\mathbf{ss}_s$. This means that, if KEM_s is IND-CCA secure, then no party without the long-term secret key $\mathbf{sk}_s$ can correctly obtain the AEAD secret key. In other words, the early application data is **implicitly authenticated**. While this provides less security than explicitly authenticated application data, to this day there is no known attack exploiting the implicit authentication. In addition, explicit authentication can be achieved at one more round trip, at which time KEMTLS becomes as secure as TLS 1.3 using comparable cryptographic primitives.

2.1.3 KEMTLS performance

present performance data from prior results: desktop, PDK, IoT **but will other people’s data contribute to this thesis? If not then maybe don’t just copy-paste other people’s data.**

2.2 IND-1CCA KEM

INDistinguishability under (adaptive) Chosen Ciphertext Attacks (IND-CCA) is the desired level of security for a key encapsulation mechanism. Intuitively, IND-CCA security requires that the shared secret from a random encapsulation cannot be distinguished from random bit string of equal length with non-negligible probability, even when the adversary has access to a decapsulation oracle. Theoretically speaking, the number of decapsulation queries is bounded by the requirement that the adversary is “efficient”: that is, the adversary, classical or quantum, can only have polynomial time complexity. In practice, NIST’s

PQC standardization project evaluates a candidate KEM’s CCA security under the upper bound of 2^{64} decapsulation queries[59].

However, within the context of an TLS 1.3 (and KEMTLS) key exchange, IND-CCA security is overkill. This is because TLS 1.3 is designed with forward secrecy, which requires that the keypair $(\mathbf{pk}_e, \mathbf{sk}_e) \xleftarrow{\$} \text{KEM}_e.\text{KeyGen}()$ exchanged in `ClientHello` and `ServerHello` be freshly generated at every session. Consequently, in a correct TLS 1.3 implementation, each private key will only be used to decapsulate at most one ciphertext. This naturally defines an alternative security definition called IND-1CCA, which is identical to IND-CCA except that the decapsulation oracle will answer at most one query. A similar notion of “security under one query” for Diffie-Hellman-style key exchange protocol called `snPRF-ODH` is defined in [30] and used in the security reduction of TLS 1.3 handshake protocol.

On the other hand, using IND-1CCA security brings meaningful performance improvements. A popular strategy for building an IND-CCA secure KEM is to start with an IND-CPA secure public key encryption scheme (PKE), then apply some variation of the Fujisaki-Okamoto transformation [32, 33, 43]. The key techniques for achieving chosen-ciphertext security in are *de-randomization* and *re-encryption*. While there is some contention on whether *de-randomization* hurts security [10] (which indirectly hurts performance because the same security level may require larger set of parameters), there is no doubt that *re-encryption* hurts performance. The performance penalty of re-encryption is especially pronounced with many lattice-based cryptosystems whose CPA-secure encryption sub-routine is much slower than the CPA-secure decryption sub-routine. As we will show in the following section, IND-1CCA constructions can preserve randomization of the underlying PKE while not requiring re-encryption.

In Section 2.2.1 we will present an original IND-1CCA construction that transforms a CPA-secure PKE into an IND-1CCA secure KEM. Section 2.2.2 then surveys and discusses other constructions.

2.2.1 The encrypt-then-MAC transformation

Let $\mathcal{B}^*$ denote the set of finite bit strings. Let $\mathcal{K}_{\text{KEM}}$ denote the set of all possible shared secrets. Let $(\text{KeyGen}_{\text{PKE}}, \text{Enc}_{\text{PKE}}, \text{Dec}_{\text{PKE}})$ be a PKE defined over message space $\mathcal{M}_{\text{PKE}}$ and ciphertext space $\mathcal{C}_{\text{PKE}}$. Let $\text{MAC} : \mathcal{K}_{\text{MAC}} \times \mathcal{B}^* \rightarrow \mathcal{T}$ be a MAC over key space $\mathcal{K}_{\text{MAC}}$ and tag space $\mathcal{T}$. Let $G : \mathcal{B}^* \rightarrow \mathcal{K}_{\text{MAC}}, H : \mathcal{B}^* \rightarrow \mathcal{K}_{\text{KEM}}$ be hash functions. The encrypt-then-MAC transformation $\text{ETM}[\text{PKE}, \text{MAC}, G, H]$ constructs a KEM $(\text{KeyGen}_{\text{ETM}}, \text{Encap}_{\text{ETM}}, \text{Decap}_{\text{ETM}})$ (Figure 2.2).

$\text{KeyGen}_{\text{ETM}}()$	$\text{Encap}_{\text{ETM}}(\text{pk})$	$\text{Decap}_{\text{ETM}}(\text{sk}, c)$
1: $(\text{pk}, \text{sk}) \xleftarrow{\$} \text{KeyGen}_{\text{PKE}}()$ 2: $s \xleftarrow{\$} \mathcal{M}_{\text{PKE}}$ 3: $\text{sk} \leftarrow (\text{sk}, s)$ 4: return (pk, sk)	1: $m \xleftarrow{\$} \mathcal{M}_{\text{PKE}}$ 2: $k \leftarrow G(m)$ 3: $c' \xleftarrow{\$} \text{Enc}_{\text{PKE}}(\text{pk}, m)$ 4: $t \leftarrow \text{MAC}(k, c')$ 5: $c \leftarrow (c', t)$ 6: $K \leftarrow H(m, c)$ 7: return (c, K)	Require: $c = (c', t)$ Require: $\text{sk} = (\text{sk}', s)$ 1: $\hat{m} \leftarrow \text{Dec}_{\text{PKE}}(\text{sk}', c')$ 2: $\hat{k} \leftarrow G(\hat{m})$ 3: if $\text{MAC}(\hat{k}, c') = t$ then 4: $K \leftarrow H(\hat{m}, c)$ 5: else 6: $K \leftarrow H(s, c)$ 7: end if 8: return K

Figure 2.2: The encrypt-then-MAC KEM routines

We chose to construct KEM_{ETM} using implicit rejection $K \leftarrow H(s, c)$: on invalid ciphertexts, the decapsulation routine returns a fake shared secret that depends on the ciphertext and some secret values, though choosing to use explicit rejection should not impact the security of the KEM. In addition, because the underlying PKE can be randomized, the shared secret $K \leftarrow H(m, c)$ must depend on both the plaintext and the ciphertext. According to [25, 43], if the input PKE is *rigid* (i.e. $m = \text{Dec}(\text{sk}, c)$ if and only if $c = \text{Enc}(\text{pk}, m)$, such as with RSA), then the shared secret may be derived from the plaintext alone $K \leftarrow H(m)$.

The CCA security of KEM_{ETM} can be intuitively argued through an adversary's inability to learn additional information from the decapsulation oracle. For an adversary A to produce a valid tag for some unauthenticated ciphertext c' , it must either know the correct symmetric key or produce a forgery. Under the Random Oracle Model (ROM), A cannot know the symmetric key without knowing its pre-image under the hash function G , so A must either produced c' honestly, or have broken the one-wayness of the underlying PKE. This means that the decapsulation oracle will not leak information on decryption that the adversary does not already know. We formalize the security in Theorem 2.2.1

Theorem 2.2.1. *For every IND-qCCA adversary A against KEM_{ETM} making q decapsulation queries, there exists a OW-PCA adversary B against the underlying PKE making at least q decapsulation queries, and an existential forgery adversary C against the underlying MAC such that:*

$$\text{Adv}_{\text{KEM}_{\text{ETM}}}^{\text{IND-CCA}}(A) \leq q \cdot \text{Adv}_{\text{MAC}}(C) + 2 \cdot \text{Adv}_{\text{PKE}}^{\text{OW-PCA}}(B)$$

From here, we can reduce OW-PCA security to OW-CPA security using a guessing argument: a CPA adversary can simulate a PCA oracle simply by randomly sampling from $\{0, 1\}$ as response to every plaintext-checking query. Thus, the probability that all guesses are correct and PCA adversary retains its advantage is at most 2^{-q} . In other words:

Lemma 2.2.2. *For every efficient q -query OW-PCA adversary A against some PKE, there exists some OW-CPA adversary B such that*

$$\mathit{Adv}_{PKE}^{\text{OW-PCA}}(A) \leq 2^q \cdot \mathit{Adv}_{PKE}^{\text{OW-CPA}}(B)$$

Proof of Theorem 2.2.1

We will prove Theorem 2.2.1 using a sequence of game. A summary of the the sequence of games can be found in Figure 2.3 and 2.4. From a high level we made three incremental modifications to the IND-CCA game for KEM_{ETM} :

1. Replace the true decapsulation oracle with a simulated decapsulation oracle. The simulated decapsulation oracle does not directly decrypt the queried ciphertext. Instead it searches through the hash oracle and looks for matching queries using the plaintext-checking oracle. The true decapsulation oracle and the simulated decapsulation oracle disagree if and only if the adversary queries with a ciphertext that contains a forged MAC tag, so the adversary cannot distinguish the two games more than it can perform existential forgery against the underlying MAC.
2. Replace the pseudorandom MAC key $k^* \leftarrow G(m^*)$ with a uniformly random MAC key $k^* \xleftarrow{\$} \mathcal{K}_{\text{MAC}}$. Under ROM, the adversary cannot distinguish this game from the previous one unless it queries the hash oracle with m^* , in which case a second adversary with access to a plaintext-checking oracle can win the OW-PCA game against the underlying PKE.
3. Replace the pseudorandom shared secret $K_0 \leftarrow H(m^*, c)$ with a truly random shared secret $K_0 \xleftarrow{\$} \mathcal{K}_{\text{KEM}}$. Similar to the point above, the adversary cannot distinguish the change more than it can break the one-wayness of the underlying PKE. Furthermore, since both K_0 and K_1 are uniformly random, no adversary can have any advantage.

A OW-PCA adversary can then simulate the modified IND-CCA game for the KEM adversary, and the advantage of the OW-PCA adversary is associated with the probability of certain behaviors of the KEM adversary.

IND-CCA game for KEM_{ETM}	Decap oracle $\mathcal{O}^{\text{Decap}}(c)$
1: $(\text{pk}, \text{sk}) \xleftarrow{\$} \text{KeyGen}_{\text{ETM}}()$ 2: $m^* \xleftarrow{\$} \mathcal{M}$ 3: $c' \xleftarrow{\$} \text{Enc}_{\text{PKE}}(\text{pk}, m^*)$ 4: $k^* \leftarrow G(m^*)$ $\triangleright$ Game 0-1 5: $k^* \xleftarrow{\$} \mathcal{K}_{\text{MAC}}$ $\triangleright$ Game 2-3 6: $t \leftarrow \text{MAC}(k^*, c')$ 7: $c^* \leftarrow (c', t)$ 8: $K_0 \leftarrow H(m^*, c^*)$ $\triangleright$ Game 0-2 9: $K_0 \xleftarrow{\$} \mathcal{K}_{\text{KEM}}$ $\triangleright$ Game 3 10: $K_1 \xleftarrow{\$} \mathcal{K}_{\text{KEM}}$ 11: $b \xleftarrow{\$} \{0, 1\}$ 12: $\hat{b} \leftarrow A^{\mathcal{O}^{\text{Decap}}}(\text{pk}, c^*, K_b)$ $\triangleright$ Game 0 13: $\hat{b} \leftarrow A^{\mathcal{O}_1^{\text{Decap}}}(\text{pk}, c^*, K_b)$ $\triangleright$ Game 1-3 14: return $\llbracket \hat{b} = b \rrbracket$	1: $(c', t) \leftarrow c$ 2: $\hat{m} = \text{Dec}_{\text{PKE}}(\text{sk}', c')$ 3: $\hat{k} \leftarrow G(\hat{m})$ 4: if $\text{MAC}(\hat{k}, c') = t$ then 5: $K \leftarrow H(\hat{m}, c)$ 6: else 7: $K \leftarrow H(z, c)$ 8: end if 9: return K
Hash oracle $\mathcal{O}^G(m)$	$\mathcal{O}_1^{\text{Decap}}(c)$
1: if $\exists(\tilde{m}, \tilde{k}) \in \mathcal{L}^G : \tilde{m} = m$ then 2: return $\tilde{k}$ 3: end if 4: $k \xleftarrow{\$} \mathcal{K}_{\text{MAC}}$ 5: $\mathcal{L}^G \leftarrow \mathcal{L}^G \cup \{(m, k)\}$ 6: return k	1: $(c', t) \leftarrow c$ 2: if $\exists(\tilde{m}, \tilde{k}) \in \mathcal{L}^G : \tilde{m} = \text{Dec}_{\text{PKE}}(\text{sk}', c') \wedge \text{MAC}(\tilde{k}, c') = t$ then 3: $K \leftarrow H(\tilde{m}, c)$ 4: else 5: $K \leftarrow H(z, c)$ 6: end if 7: return K
	$\mathcal{O}^H(m, c)$
	1: if $\exists(\tilde{m}, \tilde{c}, \tilde{K}) \in \mathcal{L}^H : \tilde{m} = m \wedge \tilde{c} = c$ then 2: return $\tilde{K}$ 3: end if 4: $K \xleftarrow{\$} \mathcal{K}_{\text{KEM}}$ 5: $\mathcal{L}^H \leftarrow \mathcal{L}^H \cup \{(m, c, K)\}$ 6: return K

Figure 2.3: Sequence of games in the proof of Theorem 2.2.1

$B(\text{pk}, c'^*)$ 1: $z \xleftarrow{\$} \mathcal{M}$ 2: $k \xleftarrow{\$} \mathcal{K}_{\text{MAC}}$ 3: $t \leftarrow \text{MAC}(k, c'^*)$ 4: $c^* \leftarrow (c'^*, t)$ 5: $K \xleftarrow{\$} \mathcal{K}_{\text{KEM}}$ 6: $\hat{b} \leftarrow A^{\mathcal{O}_B^{\text{Decap}}, \mathcal{O}_B^G, \mathcal{O}_B^H}(\text{pk}, c^*, K)$ 7: if $\text{ABORT}(m)$ then 8: return m 9: end if	$\mathcal{O}_B^{\text{Decap}}(c)$ 1: $(c', t) \leftarrow c$ 2: if $\exists(\tilde{m}, \tilde{k}) \in \mathcal{L}^G : \mathcal{O}^{\text{PCO}}(\tilde{m}, c') = 1 \wedge \text{MAC}(\tilde{k}, c') = t$ then 3: $K \leftarrow H(\tilde{m}, c)$ 4: else 5: $K \leftarrow H(z, c)$ 6: end if 7: return K
$\mathcal{O}_B^H(m, c)$ if $\mathcal{O}^{\text{PCO}}(m, c'^*) = 1$ then $\text{ABORT}(m)$ end if if $\exists(\tilde{m}, \tilde{c}, \tilde{K}) \in \mathcal{L}^H : \tilde{m} = m \wedge \tilde{c} = c$ then return $\tilde{K}$ end if $K \xleftarrow{\$} \mathcal{K}_{\text{KEM}}$ $\mathcal{L}^H \leftarrow \mathcal{L}^H \cup \{(m, c, K)\}$ return K	$\mathcal{O}_B^G(m)$ 1: if $\mathcal{O}^{\text{PCO}}(m, c'^*) = 1$ then 2: $\text{ABORT}(m)$ 3: end if 4: if $\exists(\tilde{m}, \tilde{k}) \in \mathcal{L}^G : \tilde{m} = m$ then 5: return $\tilde{k}$ 6: end if 7: $k \xleftarrow{\$} \mathcal{K}_{\text{MAC}}$ 8: $\mathcal{L}^G \leftarrow \mathcal{L}^G \cup \{(m, k)\}$ 9: return k

Figure 2.4: OW-PCA adversary B simulates game 3 for IND-CCA adversary A in the proof for Theorem 2.2.1

Proof. *Game 0* is the standard KEM IND-CCA game. The decapsulation oracle $\mathcal{O}^{\text{Decap}}$ executes the decapsulation routine using the challenge keypair and return the results faithfully. The queries made to the hash oracles $\mathcal{O}^G, \mathcal{O}^H$ are recorded to their respective tapes $\mathcal{L}^G, \mathcal{L}^H$.

Game 1 is identical to game 0 except that the true decapsulation oracle $\mathcal{O}^{\text{Decap}}$ is replaced with a simulated oracle $\mathcal{O}_1^{\text{Decap}}$. Instead of directly decrypting c' as in the decapsulation routine, the simulated oracle searches through the tape $\mathcal{L}^G$ to find a matching query $(\tilde{m}, \tilde{k})$ such that $\tilde{m}$ is the decryption of c' . The simulated oracle then uses $\tilde{k}$ to validate the tag t against c' .

If the simulated oracle accepts the queried ciphertext as valid, then there is a matching query that also validates the tag, which means that the queried ciphertext is honestly generated. Therefore, the true oracle must also accept the queried ciphertext. On the other hand, if the true oracle rejects the queried ciphertext, then the tag is simply invalid under the MAC key $k = G(\text{Dec}(\text{sk}', c'))$. Therefore, there could not have been a matching query that also validates the tag, and the simulated oracle must also rejects the queried ciphertext.

This means that from the adversary A 's perspective, game 1 and game 0 differ only when the true oracle accepts while the simulated oracle rejects, which means that t is a valid tag for c' under $k = G(\text{Dec}(\text{sk}', c'))$, but k has never been queried. Under the random oracle model, such k is a uniformly random sample of $\mathcal{K}_{\text{MAC}}$ that the adversary does not know, so for A to produce a valid tag is to produce a forgery against the MAC under an unknown and uniformly random key. Therefore, we can bound the probability that the true decapsulation oracle disagrees with the simulated oracle by the probability that some MAC adversary produces a forgery:

$$P[\mathcal{O}^{\text{Decap}}(c) \neq \mathcal{O}_1^{\text{Decap}}(c)] \leq \text{Adv}_{\text{MAC}}(C).$$

Across all q decapsulation queries, the probability that at least one query is a forgery is thus at most $q \cdot P[\mathcal{O}^{\text{Decap}}(c) \neq \mathcal{O}_1^{\text{Decap}}(c)]$. By the difference lemma:

$$|\text{Adv}_{G_0}(A) - \text{Adv}_{G_1}(A)| \leq q \cdot \text{Adv}_{\text{MAC}}(C).$$

Game 2 is identical to game 1, except that the challenger samples a uniformly random MAC key $k^* \xleftarrow{\$} \mathcal{K}_{\text{MAC}}$ instead of deriving it from m^* . From A 's perspective the two games are indistinguishable, unless A queries G with the value of m^* . Denote the probability that A queries G with m^* by $P[\text{QUERY } G]$, then:

$$|\text{Adv}_{G_1}(A) - \text{Adv}_{G_2}(A)| \leq P[\text{QUERY } G].$$

Game 3 is identical to game 2, except that the challenger samples a uniformly random shared secret $K_0 \xleftarrow{\$} \mathcal{K}_{\text{KEM}}$ instead of deriving it from m^* and t . From A 's perspective the two games are indistinguishable, unless A queries H with $(m^*, \cdot)$. Denote the probability that A queries H with $(m^*, \cdot)$ by $P[\text{QUERY } H]$, then:

$$|\text{Adv}_{G_2}(A) - \text{Adv}_{G_3}(A)| \leq P[\text{QUERY } H].$$

Since in game 3, both K_0 and K_1 are uniformly random and independent of all other variables, no adversary can have any advantage: $\text{Adv}_{G_3}(A) = 0$.

We will bound $P[\text{QUERY } G]$ and $P[\text{QUERY } H]$ by constructing a OW-PCA adversary B against the underlying PKE that uses A as a sub-routine. B 's behaviors are summarized in Figure 2.4.

B simulates game 3 for A : upon receiving the public key pk and challenge encryption c'^* , B samples random MAC key and session key to produce the challenge encapsulation, then feeds it to A . When simulating the decapsulation oracle, B uses the plaintext-checking oracle to look for matching queries in $\mathcal{L}^G$. When simulating the hash oracles, B uses the plaintext-checking oracle to detect when $m^* = \text{Dec}(\text{sk}', c'^*)$ has been queried. When m^* is queried, B terminates A and returns m^* to win the OW-PCA game. In other words:

$$\begin{aligned} P[\text{QUERY } G] &\leq \text{Adv}_{\text{PKE}}^{\text{OW-PCA}}(B), \\ P[\text{QUERY } H] &\leq \text{Adv}_{\text{PKE}}^{\text{OW-PCA}}(B). \end{aligned}$$

Combining all equations above produce the desired security bound. $\square$

2.2.2 Related works

To our knowledge, the concept of IND-qCCA security is first published by Cramer et al. [23], whose proposed a generic black-box IND-qCCA PKE construction whose security reduces to the IND-CPA security of the underlying PKE and the existential unforgeability of the underlying one-time signature [53]. It is particularly remarkable how this reduction is proved in the standard model instead of the random oracle model, although this came at the cost of computational efficiency.

KeyGen	Encap(pk)	Decap(sk, c)
1: $(\mathbf{pk}, \mathbf{sk}) \xleftarrow{\$} \text{KeyGen}^{\text{PKE}}()$	1: $m \xleftarrow{\$} \mathcal{M}^{\text{PKE}}$ 2: $c \xleftarrow{\$} \text{Enc}^{\text{PKE}}(\mathbf{pk}, m)$ 3: $K \leftarrow H(m, c)$ 4: return (c, K)	1: $\hat{m} \leftarrow \text{Dec}^{\text{PKE}}(\mathbf{sk}, c)$ 2: if $\hat{m} = \perp$ then 3: return $\perp$ 4: end if 5: return $H(\hat{m}, c)$

Figure 2.5: T_H transformation

Huguenin-Dumittan and Vaudenay [45] extended this line of work to the post-quantum setting by proposing two IND-qCCA KEM constructions (respectively denoted by T_{CH} and T_H) from CPA-secure PKEs. T_{CH} achieves qCCA security using a “confirmation hash”: the KEM ciphertext includes the PKE ciphertext and a hash of the PKE plaintext, and the KEM shared secret is derived from hashing the PKE plaintext (the encrypt-then-MAC transformation can be seen as a special case of the T_{CH} transformation not sure if we should mention this similarity). The security reduction is loose by a factor of 2^q where q is the number of decapsulation queries. T_H achieves qCCA security by deriving the shared secret from hashing both the PKE plaintext and ciphertext (Figure 2.5), so decapsulation queries with modified ciphertext will be answered with independent random values under the random oracle model. T_H ’s construction is simpler than T_{CH} and retains the original ciphertext length, but on the other hand, the security reduction is looser, with a factor of q_H^{2q} , where q_H is the number of hash oracle queries, and q is the number of decapsulation queries. The authors also proved qCCA security of both transformations under quantum random oracle model (QROM).

In the same paper, the two authors also observed that in TLS 1.3, the handshake secret (and thus all subsequent secrets and keys) is derived with input from the handshake transcript, which means that the KEM public key and ciphertext, embedded in **ClientHello** and **ServerHello** respectively, are already hashed into the handshake secret. This is similar to the T_H construction where the shared secret is hashed from PKE plaintext and ciphertext, and indeed the authors proved that TLS 1.3 remains multi-stage secure even if the KEM is only CPA secure. The initial proof presented by the authors had a looseness factor of $O(q^6)$, which was later improved by Zhou et al.[88] to $O(q)$ for randomized PKEs, and $O(1)$ for rigid PKEs.

Chapter 3

Implementation

To evaluate post-quantum TLS 1.3 and KEMTLS on constrained devices, we implemented post-quantum TLS 1.3 and KEMTLS on top of WolfSSL, a TLS library written in the C programming language. We then implemented a minimal TLS client on the Raspberry Pi Pico 2 W, a microcontroller with two ARM Cortex-M33 cores ¹ and 512kB of SRAM, and measures the time it takes for the Pico client to complete a TLS 1.3 or KEMTLS handshake with a server. This chapter describes some of the implementation details, the benchmarking methodology, and the performance measurements.

3.1 WolfSSL

WolfSSL is a modern open-source TLS library written in C. In addition to a complete TLS stack, WolfSSL also includes its own cryptography library called WolfCrypt. Both WolfSSL and WolfCrypt are optimized for code size, speed, and memory footprint, and its portability and ease of configuration greatly simplifies managing multiple build targets using a single code base.

3.1.1 Integrating additional post-quantum algorithms

As of June 2025, WolfCrypt contains an in-house implementation of ML-KEM and ML-DSA. Both implementation are skillfully optimized, achieving at least 2x speedup on the

¹The Pico 2 W also has 2 RISC-V cores, though we did not use them in this project.

Pico compared to the reference implementation ². Unfortunately, this makes the comparison across different schemes unfair. Instead, I chose to integrate with PQClean’s clean implementations.

While PQClean is not specifically optimized for embedded system builds, all of its clean implementations are trivially portable to ARM. One non-trivial challenge is adapting the `randombytes` API in PQClean to an embedded system build with no operating system. Fortunately, WolfCrypt’s `WC_RNG` API provides a common abstraction that works on both a desktop build (where random bits can be sourced from `/dev/urandom`) and Pico (where random bits are collected from various peripherals by the SDK). In the end, we expanded the `randombytes` API so it can be told to source random bits from user specified instance of `WC_RNG`.

Expanding the selection of KEMs for the initial key exchange (i.e. `ClientHello` and `ServerHello`) is straightforward, thanks to the fact that WolfSSL already supports ML-KEM for key exchange. The `NamedGroup` enum is trivially captured using a single 16-bit integer, and the logic for branching into the correct KEM allows for a simple `switch-case` block.

Expanding the selection of post-quantum signatures is trivial thanks to previous efforts to integrate `liboqs` into WolfSSL. We only need to replace all uses of `liboqs` with their equivalents in PQClean.

3.1.2 Implementing KEMTLS

Generating certificate chain and private keys

WolfCrypt’s `asn.h` API provides a nearly complete collection of tools needed to generate certificate chains, encode certificates and private keys according to DER, then further encode them to PEM format. At the time of writing this thesis, WolfCrypt does not support signing a certificate signing request (CSR), but for benchmarking purposes I control the entire chain, and WolfCrypt does support directly signing the body of a certificate.

Modifying WolfCrypt’s `asn.h` module to support KEM public key in a certificate is relatively straightforward. The only non-trivial obstacle comes from how WolfSSL handles OIDs. Object Identifier (OID) is a variable-length sequence of integers used to identify individual cryptographic primitives. For example, the OID for ML-KEM-512 is

²The Keccak implementation in WolfCrypt is only roughly 10% faster than PQClean’s implementation, so the optimization must have come elsewhere

2.16.840.1.101.3.4.4.1. OID is included in an certificate to identify the public key and the signature; it is also included in the DER encoding of private keys. Having variable length makes OID tedious to work with when programming in C: unlike `NamedGroup`, which has fixed length that can be captured in a 16-bit integer, OID cannot be easily abstracted using a fixed-sized enum type. WolfSSL works around this limitation by using an “OID sum” algorithm, which computes an “hash” of an OID that fit into a 32-bit integer. Compressing variable-length integer sequence into a 32-bit integer carries with it the risk of collision, and in fact the first version of the OID summing algorithm indeed ran into a collision between SPHINCS-192-fast and SPHINCS-128-fast. A newer OID summing algorithm provided stronger collision resistance and resolved this issue. All OID sums are stored in a header file `oid_sum.h`, which is generated by a Perl script.

Implementing unilaterally authenticated KEMTLS handshake

KEMTLS handshake workflow is identical to TLS 1.3’s handshake flow from the beginning until client starts processing server’s `Certificate` message. Even after KEMTLS and TLS 1.3’s handshake flow diverges, they still share the format of the `Finished` message (which contains exactly one HMAC tag). Finally, once the handshake is complete, TLS 1.3 and KEMTLS exchange application data in identical fashion. The similarity between TLS 1.3 and KEMTLS handshake workflow allows us to reuse a significant part of WolfSSL’s TLS 1.3 implementation, diverging at only a handful of places that are easy to reason about. While working with a TLS library written in C is intimidating at first, this implementation strategy proved successful, and I was able to finish implementing KEMTLS in less than a month using only around 4600 lines of code change.

The `WOLFSSL` struct is used on both client-side and server-side and encodes the pair of client and server state as a global TLS state. We begin modifying the TLS state machine by adding two flags `haveMlKemAuth` and `haveHqcAuth` to the main `WOLFSSL` struct. In a unilaterally authenticated KEMTLS handshake, the two flags are set on the server side when the server loads a KEM private key. Detecting a KEM private key is cleanly accomplished because at certificate generation, private keys are encoded according to DER, and the OID of the KEM scheme is included. If the OID belongs to one of ML-KEM’s variants, then `haveMlKemAuth` is set, and if the OID belongs to one of HQC’s variants, then `haveHqcAuth` is set. On the client side, these two flags are set when the client finds a ML-KEM or HQC public key in the certificate chain sent by the server. The combination of these two flags is sufficient for deciding when the two peers are performing a KEMTLS or TLS 1.3 handshake, and all divergence between KEMTLS and TLS 1.3 handshake flow will be controlled by these two flags.

On the client side, KEMTLS and TLS 1.3 handshake flows first diverge after the client finishes processing server's **Certificate** message. In signature-based TLS 1.3, client's immediate next step is to receive and process server's **CertificateVerify** containing a signature over the handshake transcript. In KEMTLS, client will not receive additional message. Instead, it uses the KEM public key to encapsulate the **authentication secret**, then sends the ciphertext to the server in a **KemCiphertext** message. We followed the original KEMTLS implementation's **KemCiphertext** format as a handshake message whose payload contains the raw ciphertext and no additional metadata. Within the context of this project, each server instance will only load one private key, so **KemCiphertext** not carrying metadata on the ciphertext will not cause confusion. However, WolfSSL supports loading multiple private keys for authentication, in which case it might be necessary for **KemCiphertext** to carry metadata such as an OID.

Client's **KemCiphertext** is encrypted under handshake traffic keys derived from the unauthenticated handshake secret (HS). After sending **KemCiphertext**, client must update the key schedule by mixing in the authentication secret and deriving the authenticated handshake secret (AHS). From AHS, the client will derive new handshake traffic key for encrypting additional handshake messages, as well as application traffic key. This key schedule update is necessary for subsequent messages to provide implicit authentication: no adversary, even if it compromises the handshake secret, can decrypt subsequent handshake message or application data without the long-term secret key.

Client's **Finished** follows the same format as TLS 1.3's **Finished**: a handshake message whose payload contains a raw HMAC tag computed under the **finished_key** against the handshake transcript. The MAC key is derived from the **MasterSecret**, which is itself derived from AHS. After sending **Finished**, the client can start sending application data without receiving server's **Finished** first. Because application data will be encrypted under application traffic key derived from authenticated handshake secret, any server successfully decrypting them is implicitly authenticated. Finally, client explicitly authenticates the server after receiving server's **Finished**.

On the server side, KEMTLS and TLS 1.3 handshake flow first diverges after server finishes sending its **Certificate**. If either of **haveMlKemAuth** and **haveHqcAuth** flag is set, then after exiting **SendTls13Certificate**, instead of constructing and sending **CertificateVerify**, server will receive and process **KemCiphertext**. After decapsulating client's **KemCiphertext**, server similarly needs to update the key schedule: first derive the authenticated handshake secret using the authentication secret, then derive new handshake traffic keys, HMAC keys for processing client's **Finished** and constructing server's **Finished**, and finally the application traffic keys. Last but not least, server needs to construct and send its **Finished** before it can start sending application data.

In our implementation, we used the handshake message type for `client_key_exchange` when constructing `KemCiphertext`. `client_key_exchange` is a handshake message type that exists only in TLS 1.2 or prior but not in TLS 1.3. This upsets some sanity checks in WolfSSL’s TLS 1.3 state machine. Similarly, the different order of messages, particularly the order of `Finished`, can also cause the same set of sanity checks to fail. For the purpose of benchmarking performance only, we modified these sanity checks to ignore message orders when `haveMlKemAuth` or `haveHqcAuth` is set. However, in production use, KEMTLS will definitely require a distinct set of checks to ensure the integrity of the handshake state machine.

3.1.3 Miscellaneous comments

We appreciate the many thoughtful design choices made in both WolfSSL and WolfCrypt. Using C preprocessing macros defined in a easily manageable `user_settings.h` file allowed us to build for three different platforms (Apple Silicon on the author’s laptop running MacOS, `x86_64` on the test server running Linux, and 32-bit ARM on baremetal) using a single codebase. The modular I/O callback API made it possible to run WolfSSL on the Pi Pico without any RTOS. WolfSSL’s repository even provided ready-made integration with the Pico-SDK so that using Pico’s hardware RNG with WolfCrypt’s `WC_RNG` struct requires no additional effort from the authors.

Another difficulty with WolfSSL comes from the need for manual memory management. This is especially the case when building for the Pico, which has only 512kB SRAM and can be easily overwhelmed by memory leaks since post-quantum keys, ciphertexts, and signatures all consume 1-10 kilobytes of memories each. Fortunately, there are only a handful of places where WolfSSL requires dynamic memory allocations, and they are either freed within the function scope or freed at the end of the handshake as part of `wolfSSL_shutdown` or `wolfSSL_free`. Other instances of dynamic memory allocation were flawlessly managed in WolfSSL’s existing source code, allowing the Pico to continuously perform thousands of handshakes without exhausting its SRAM or needing assistance from an RTOS.

3.2 Hardware setup

We chose to test TLS 1.3 and KEMTLS performance on a Raspberry Pi Pico 2 W and a Linux desktop. The Raspberry Pi Pico 2 W is a microcontroller developed by the Raspberry Pi Foundation and released in late 2024. At the heart of the Pico 2 W is the RP2350 chip,

which contains two ARM Cortex-M33 cores and two Hazard3 RISC-V cores, though only one architecture can be enabled at boot time. The chip is clocked at 150MHz. The Pico 2 W also has 520KB of SRAM and 4MB of flash storage. For connectivity, the Pico 2 W has a CYW43439 module that supports 2.4 GHz Wifi and Bluetooth 5.4.

For developing on the Pico 2 W, we used the GNU ARM toolchain (`arm-none-eabi-gcc` version 8.5.0) and the Pico-SDK, which comes with a standard library as well as integration with third-party open source projects, most notably `lwip` (lightweight IP). `lwip` is a TCP/IP library designed for embedded system, and the Pico-SDK provided tight integration between `lwip` and the `cyw43` driver, which allowed us to implement a simple TCP stream on top of the raw TCP API. We also used `lwip` to implement DNS and time synchronization over NTP, the latter being necessary for WolfSSL to validate the expiration dates of peer's certificates.

Because of the lack of a file system, certificates must be baked into the firmware at compilation. While it is possible to define certificates as raw bytes in C preprocessing macro, we found it much easier to encode the certificates in PEM format, which consists entirely of ASCII characters. On the other hand, while private keys can also be encoded in DER or PEM format and included into the firmware at compilation, without further protection they can be easily extracted from the devices via a firmware dump. The Pico 2 W features a one-time programmable (OTP) ROM that, once enabled and written to, will permanently be protected by secure boot. In December 2024, Raspberry Pi Ltd. launched a hacking challenge³ in which the participants try to extract a secret from a secure-boot-enabled Pico 2 W. While some participants have successfully extracted the secret, all publicly known winners used intrusive methods that required extensive physical access and non-trivial external hardware. Another issue with using the OTP for private key is size, since on RP2350 the OTP has a limited capacity of 8KB, although it is possible to store a seed that can then be expanded into the private key at bootup. Embedded devices that need to store private keys should consider using a peripheral device, such as a trusted platform module (TPM) or secure hardware module (HSM).

The Linux desktop has an 8-core AMD Ryzen 7 1700X with 12GB of RAM, running Ubuntu 24.04 LTS, and compiling with `GCC 13.3.0`.

Our network setup takes advantage of Pico 2 W's wireless capabilities. We used a netowrk router that is connected to University of Waterloo's internal network via Ethernet, then connect the Pico 2 W to the router via Wifi. The Linux desktop was connected to the internal network via Ethernet as well. **I need more metrics on the network :(**

³https://github.com/raspberrypi/rp2350_hacking_challenge

Chapter 4

Performance of post-quantum TLS 1.3 and KEMTLS

In this section we will compare the performance of post-quantum TLS 1.3, KEMTLS, and classical TLS 1.3.

4.1 Communication cost

TLS 1.3 recommended ephemeral elliptic curve Diffie-Hellman key agreement (ECDHE) to be the default scheme for key exchange in `ClientHello` and `ServerHello`. Using ECDHE, the client and the server each sends the other an element from the agreed group of points on an elliptic curve, so round-trip communication cost is the sum of two group points. When using a post-quantum KEM, the client sends to the server an encapsulation key, and the server sends back a ciphertext obtained under said encapsulation key. The round-trip communication cost of a KEM key agreement is thus the sum of a public key and a ciphertext. Table 4.1 lists the costs of each key agreement scheme.

Post-quantum digital signature schemes also have larger sizes than elliptic-curve signatures and RSA-signatures. Table 4.2 compares the sizes of public key and signature across the chosen set of signature schemes.

In our handshake setup, server's public key sits at the top of a chain of three certificates (root, intermediate, and leaf). However, the root certificate is compiled into the client's firmware, so it is not transmitted in the handshake. Table 4.3 compares the sizes of certificate chain.

Table 4.1: Key agreement communication costs (bytes).

Category	Scheme	Public key	Ciphertext	Total
ECDHE	SECP256R1	33/65		66/130
	SECP384R1	49/97		98/194
	SECP521R1	67/133		134/266
	X25519	32		64
	X448	56		112
PQC	ML-KEM-512	800	768	1568
	ML-KEM-768	1184	1088	2272
	ML-KEM-1024	1568	1568	3136
	HQC-128	2249	4433	6682
	HQC-192	4522	9029	13551
	HQC-256	7245	14469	21714

Table 4.2: Digital signatures communication costs (bytes)

Category	Scheme	Public key	Signature	Total
Classical	RSA-2048 (PKCS#1 v1.5)	256	256	512
	ECDSA SECP256R1	64	64	128
	ECDSA SECP384R1	96	96	192
	ECDSA SECP521R1	132	132	264
	Ed25519	32	64	96
	Ed448	57	114	171
PQC	ML-DSA-44	1312	2420	3732
	ML-DSA-65	1952	3309	5261
	ML-DSA-87	2592	4627	7219
	Falcon-512	897	666	1563
	Falcon-1024	1793	1280	3073
	SPHINCS+-128f	32	17088	17120
	SPHINCS+-192f	48	35664	35712
	SPHINCS+-256f	64	49856	49920
	SPHINCS+-128s	32	7856	7888
	SPHINCS+-192s	48	16224	16272
	SPHINCS+-256s	64	29792	29856

Table 4.3: Server certificate chain size (bytes). Each certificate is encoded in PEM format. For rows with one name, there are still three certificates, but they use the same scheme for all three keys.

Category	Root	Intermediate	Leaf	Size
Classical	RSA2048			2506
	ECDSA SECP256R1			1429
	ECDSA SECP384R1			1429
	ECDSA SECP521R1			1429
	Ed25519			1893
	Ed448			1893
PQ-TLS	ML-DSA-44			16763
	ML-DSA-65			???
	ML-DSA-87			???
	SPHINCS-128f	ML-DSA-44	ML-DSA-44	54734
	SPHINCS-128s	ML-DSA-44	ML-DSA-44	??
	Falcon-512	ML-DSA-44	ML-DSA-44	??
	SPHINCS-192f	ML-DSA-65	ML-DSA-65	??
	SPHINCS-192s	ML-DSA-65	ML-DSA-65	??
	SPHINCS-256f	ML-DSA-87	ML-DSA-87	??
	SPHINCS-256s	ML-DSA-87	ML-DSA-87	??
	Falcon-1024	ML-DSA-87	ML-DSA-87	??
PQ-KEMTLS	ML-DSA-44	ML-DSA-44	ML-KEM-512	???
	ML-DSA-65	ML-DSA-65	ML-KEM-768	???
	ML-DSA-87	ML-DSA-87	ML-KEM-1024	???
	ML-DSA-44	ML-DSA-44	HQC-128	???
	ML-DSA-65	ML-DSA-65	HQC-192	???
	ML-DSA-87	ML-DSA-87	HQC-256	???

4.2 Cryptographic operations performance

We benchmarked the speed of relevant cryptographic operations on the Pico 2 W. Table 4.4 and 4.5 summarizes the results.

Table 4.4: Key exchange performance on RP2350

ECDHE									
Name	KeyGen			Agree					
	Medium	P90	P99	Medium	P90	P99			
X25519									
X448									
SECP256R1									
SECP384R1									
SECP521R1									
Post-quantum KEM									
	KeyGen			Encap			Decap		
	Medium	P90	P99	Medium	P90	P99	Medium	P90	P99
ML-KEM-512									
ML-KEM-768									
ML-KEM-1024									
OT-ML-KEM-512									
OT-ML-KEM-768									
OT-ML-KEM-1024									
HQC-128									
HQC-192									
HQC-256									

4.3 Handshake latency

TLS 1.3 and KEMLS handshake can be divided into two distinct phases. The first phase is key agreement, which starts at the beginning at the handshake and ends when client finishes processing `ServerHello`. The second phase is authentication, which starts at the end of the key agreement phase and ends when the handshake is completed. We collected the timestamps at key moments from the client side:

Table 4.5: Digital signature performance on RP2350

Digital signature						
	Sign			Verify		
	Medium	P90	P99	Medium	P90	P99
Classical signatures						
RSA-2048						
Ed25519						
ECDSA (P-256)						
ECDSA (P-384)						
ECDSA (P-521)						
Post-quantum signatures						
ML-DSA-44						
ML-DSA-65						
ML-DSA-87						
SPHINCS+-128f						
SPHINCS+-128s						
SPHINCS+-192f						
SPHINCS+-192s						
SPHINCS+-256f						
SPHINCS+-256s						
Falcon-512						
Falcon-1024						

1. `ch_start`: when the handshake starts
2. `ch_sent`: when `ClientHello` is sent
3. `sh_start`: when `ServerHello` is received
4. `sh_done`: when `ServerHello` is processed
5. `auth_start`: when `Certificate` is received
6. `auth_done`: when the handshake finishes

Using these timestamps we can clearly define the boundaries of the two phases from the client’s perspective: key agreement duration is time delta between `sh_done` and `ch_start`, while authentication duration is the interval from `auth_start` to `auth_done`. These detailed timestamps also allow us to observe the impact of communication cost on handshake. For example, the interval from `sh_done` to `auth_start` reflects the time it takes for server to transmit the certificate chain.

Key agreement. We define the total key agreement time to be the length of interval from the start of the handshake to the end of processing `ServerHello`. In addition, we also track the length of time for which the Pico is doing active work: the time spent constructing `ClientHello` plus the time spent processing `ServerHello`. Table 4.6 compares the performance of various key exchange scheme using these two metrics. Figure 4.1 illustrates the distribution of data points.

Authentication. We define the *certificate transmission time* to be the length of interval from the end of key agreement to the start of processing server’s certificates, *authentication time* to be from start of processing certificates to the end of the handshake. Table 4.7 summarizes these metrics. Figure 4.2 illustrates the distribution of data points.

Finally, Table 4.8 and Figure 4.3 summarize the performance of the handshake as a whole.

4.3.1 Discussion

Category	Key agreement	Total time		CPU time	
		Median	Std	Median	Std
Classical	ECDHE (P-256)	445096.0	195745.12	438639.0	182.35
	X25519	36322.5	713456.00	28813.0	31.68
	ECDHE (P-384)				
	ECDHE (P-521)				
	X448				
PQC	ML-KEM-512	27254.0	284903.41	19443.0	85.32
	ML-KEM-768				
	ML-KEM-1024				
	OT-ML-KEM-512	16938.0	5573.21	14025.0	41.26
	OT-ML-KEM-768				
	OT-ML-KEM-1024				
	HQC-128	1341054.5	303532.68	1321492.0	80.01
	HQC-192				
	HQC-256				

Table 4.6: Comparing key agreement performance (microseconds).

Category	Certificate chain			Cert TX time	Auth time
	Root	Int	Leaf		
Classical	RSA-2048				
	ECDSA (P-256)				
	Ed25519				
PQC	ML-DSA-44				
	SPHINCS+ 128f	ML-DSA-44	ML-DSA-44		
	ML-DSA-44	ML-DSA-44	ML-KEM-512		

Table 4.7: Medium authentication durations (microseconds).

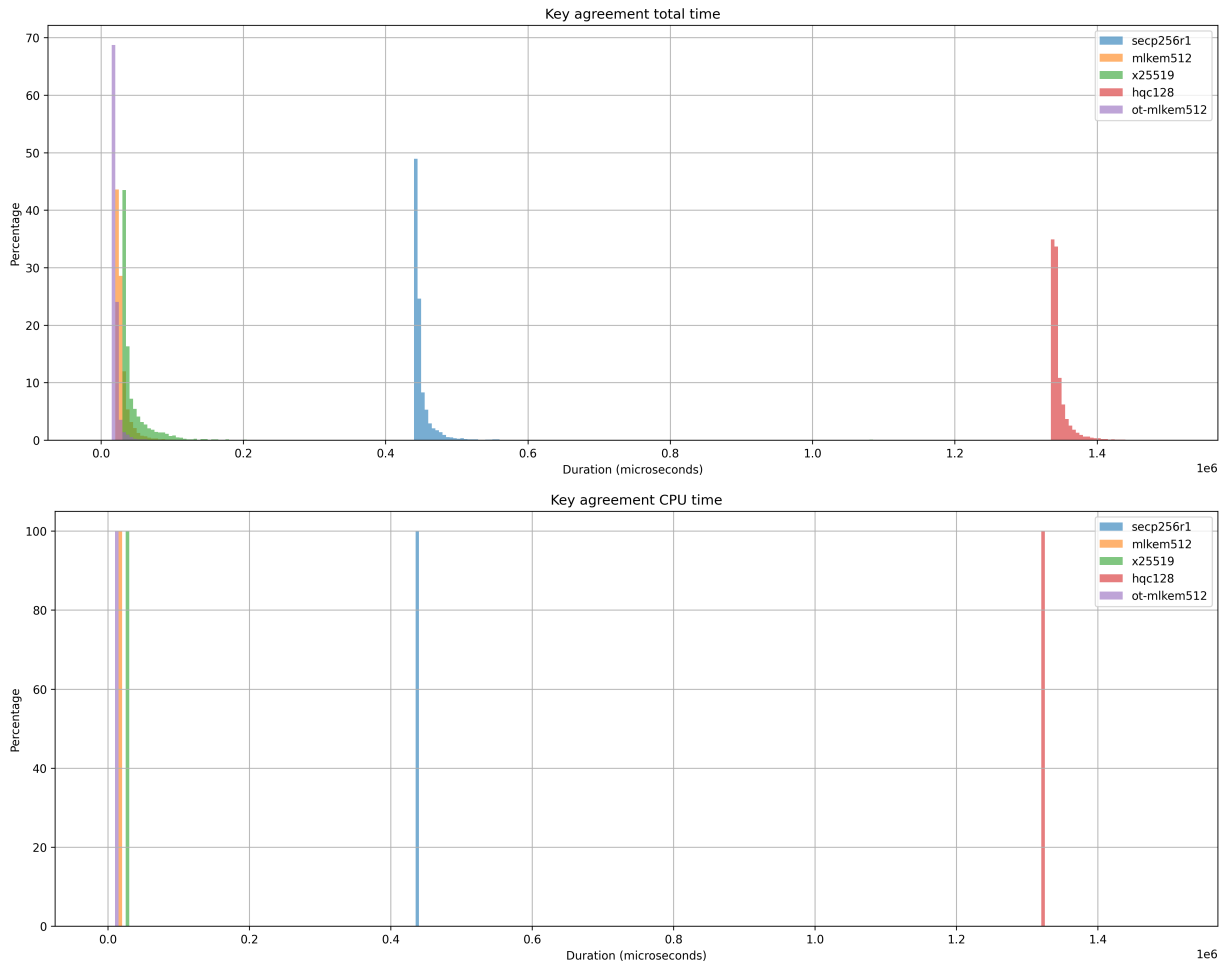


Figure 4.1: Key agreement performance (microseconds).

Category	KA	Auth			Handshake time
		Root	Int	Leaf	
Classical	ECDHE (P-256)	RSA-2048			
	ECDHE (P-256)	ECDSA (P-256)			
	X25519	Ed25519			
PQC	ML-KEM-512	ML-DSA-44			
	ML-KEM-512	SPHINCS+ 128f	ML-DSA-44	ML-DSA-44	
	ML-KEM-512	ML-DSA-44	ML-DSA-44	ML-KEM-512	

Table 4.8: Medium handshake durations (microseconds).

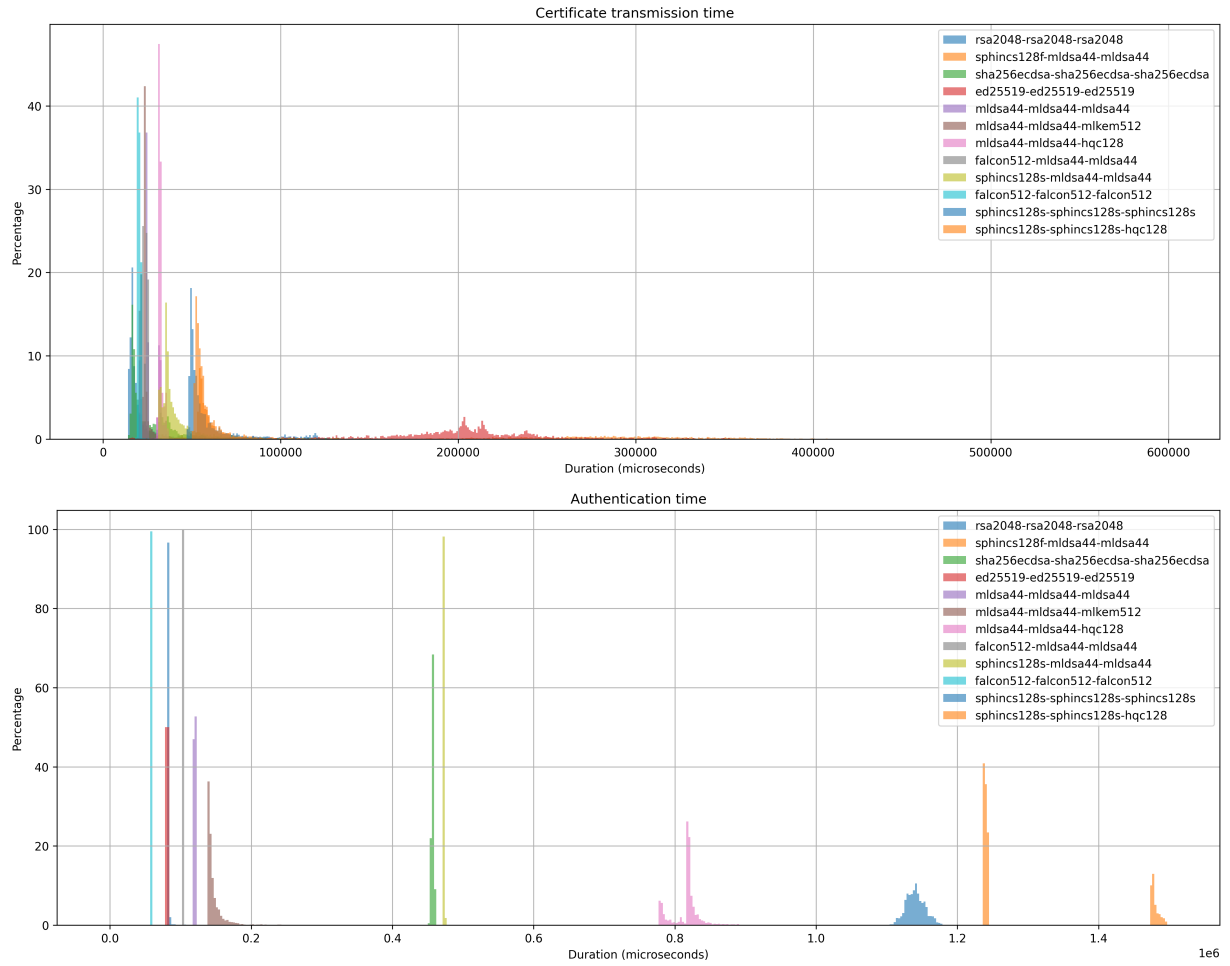


Figure 4.2: Authentication durations (microseconds).

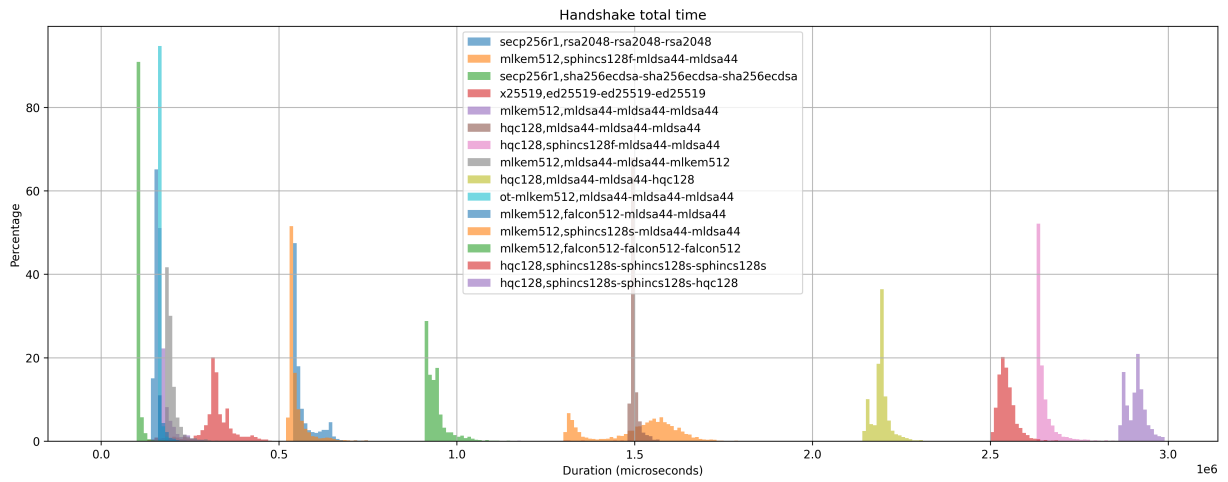


Figure 4.3: Handshake durations (microseconds).

Chapter 5

Conclusion and future works

References

- [1] Martin R. Albrecht, Christian Hanser, Andrea Höller, Thomas Pöppelmann, Fernando Virdia, and Andreas Wallner. Implementing rlwe-based schemes using an RSA co-processor. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2019(1):169–208, 2019.
- [2] Nouri Alnahawi, Johannes Müller, Jan Oupický, and Alexander Wiesmaier. Sok: Post-quantum TLS handshake. *IACR Cryptol. ePrint Arch.*, page 1873, 2023.
- [3] Nouri Alnahawi, Johannes Müller, Jan Oupický, and Alexander Wiesmaier. A comprehensive survey on post-quantum TLS. *IACR Commun. Cryptol.*, 1(2):6, 2024.
- [4] Dorian Amiet, Andreas Curiger, and Paul Zbinden. Fpga-based accelerator for post-quantum signature scheme SPHINCS-256. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(1):18–39, 2018.
- [5] Nimrod Aviram, Benjamin Dowling, Ilan Komargodski, Kenneth G. Paterson, Eyal Ronen, and Eylon Yogev. Practical (post-quantum) key combiners from one-wayness and applications to TLS. *IACR Cryptol. ePrint Arch.*, page 65, 2022.
- [6] Utsav Banerjee and Anantha P. Chandrakasan. Efficient post-quantum TLS handshakes using identity-based key exchange from lattices. In *2020 IEEE International Conference on Communications, ICC 2020, Dublin, Ireland, June 7-11, 2020*, pages 1–6. IEEE, 2020.
- [7] Utsav Banerjee, Siddharth Das, and Anantha P. Chandrakasan. Accelerating post-quantum cryptography using an energy-efficient TLS crypto-processor. In *IEEE International Symposium on Circuits and Systems, ISCAS 2020, Sevilla, Spain, October 10-21, 2020*, pages 1–5. IEEE, 2020.
- [8] Jon Barton, William J. Buchanan, Nikolaos Pitropakis, Sarwar Sayeed, and Will Abramson. Post quantum cryptography analysis of TLS tunneling on a constrained

- device. In Paolo Mori, Gabriele Lenzini, and Steven Furnell, editors, *Proceedings of the 8th International Conference on Information Systems Security and Privacy, ICISSP 2022, Online Streaming, February 9-11, 2022*, pages 551–561. SCITEPRESS, 2022.
- [9] Mihir Bellare and Phillip Rogaway. Entity authentication and key distribution. In Douglas R. Stinson, editor, *Advances in Cryptology - CRYPTO '93, 13th Annual International Cryptology Conference, Santa Barbara, California, USA, August 22-26, 1993, Proceedings*, volume 773 of *Lecture Notes in Computer Science*, pages 232–249. Springer, 1993.
 - [10] Daniel J. Bernstein. FO derandomization sometimes damages security. Cryptology ePrint Archive, Paper 2021/912, 2021.
 - [11] Daniel J. Bernstein, Billy Bob Brumley, Ming-Shing Chen, and Nicola Tuveri. Opensslntru: Faster post-quantum TLS key exchange. *CoRR*, abs/2106.08759, 2021.
 - [12] Daniel J. Bernstein, Billy Bob Brumley, Ming-Shing Chen, and Nicola Tuveri. Opensslntru: Faster post-quantum TLS key exchange. *IACR Cryptol. ePrint Arch.*, page 826, 2021.
 - [13] Daniel J. Bernstein, Billy Bob Brumley, Ming-Shing Chen, and Nicola Tuveri. Opensslntru: Faster post-quantum TLS key exchange. In Kevin R. B. Butler and Kurt Thomas, editors, *31st USENIX Security Symposium, USENIX Security 2022, Boston, MA, USA, August 10-12, 2022*, pages 845–862. USENIX Association, 2022.
 - [14] Bruno Blanchet and Charlie Jacomme. Post-quantum sound cryptoverif and verification of hybrid TLS and SSH key-exchanges. In *37th IEEE Computer Security Foundations Symposium, CSF 2024, Enschede, Netherlands, July 8-12, 2024*, pages 543–556. IEEE, 2024.
 - [15] Joppe W. Bos, Craig Costello, Michael Naehrig, and Douglas Stebila. Post-quantum key exchange for the TLS protocol from the ring learning with errors problem. *IACR Cryptol. ePrint Arch.*, page 599, 2014.
 - [16] Joppe W. Bos, Craig Costello, Michael Naehrig, and Douglas Stebila. Post-quantum key exchange for the TLS protocol from the ring learning with errors problem. In *2015 IEEE Symposium on Security and Privacy, SP 2015, San Jose, CA, USA, May 17-21, 2015*, pages 553–570. IEEE Computer Society, 2015.
 - [17] Gilles Brassard, Peter Høyer, and Alain Tapp. Quantum cryptanalysis of hash and claw-free functions. *SIGACT News*, 28(2):14–19, 1997.

- [18] Kevin Brstringhaus-Steinbach, Christoph Krau, Ruben Niederhagen, and Michael Schneider. Post-quantum TLS on embedded systems. *IACR Cryptol. ePrint Arch.*, page 308, 2020.
- [19] Kevin Brstringhaus-Steinbach, Christoph Krau, Ruben Niederhagen, and Michael Schneider. Post-quantum TLS on embedded systems: Integrating and evaluating kyber and SPHINCS+ with mbed TLS. In Hung-Min Sun, Shih-Pyng Shieh, Guofei Gu, and Giuseppe Ateniese, editors, *ASIA CCS '20: The 15th ACM Asia Conference on Computer and Communications Security, Taipei, Taiwan, October 5-9, 2020*, pages 841–852. ACM, 2020.
- [20] Fabio Campos, Jorge Chvez-Saab, Jess-Javier Chi-Domnguez, Michael Meyer, Krijn Reijnders, Francisco Rodrguez-Henrquez, Peter Schwabe, and Thom Wiggers. On the practicality of post-quantum TLS using large-parameter CSIDH. *IACR Cryptol. ePrint Arch.*, page 793, 2023.
- [21] Sofa Celi, Armando Faz-Hernndez, Nick Sullivan, Goutam Tamvada, Luke Valenta, Thom Wiggers, Bas Westerbaan, and Christopher A. Wood. Implementing and measuring KEMTLS. In Patrick Longa and Carla Rfols, editors, *Progress in Cryptology - LATINCRYPT 2021 - 7th International Conference on Cryptology and Information Security in Latin America, Bogot, Colombia, October 6-8, 2021, Proceedings*, volume 12912 of *Lecture Notes in Computer Science*, pages 88–107. Springer, 2021.
- [22] Sofa Celi, Jonathan Hoyland, Douglas Stebila, and Thom Wiggers. A tale of two models: Formal verification of KEMTLS via tamarin. In Vijayalakshmi Atluri, Roberto Di Pietro, Christian Damsgaard Jensen, and Weizhi Meng, editors, *Computer Security - ESORICS 2022 - 27th European Symposium on Research in Computer Security, Copenhagen, Denmark, September 26-30, 2022, Proceedings, Part III*, volume 13556 of *Lecture Notes in Computer Science*, pages 63–83. Springer, 2022.
- [23] Ronald Cramer, Goichiro Hanaoka, Dennis Hofheinz, Hideki Imai, Eike Kiltz, Rafael Pass, Abhi Shelat, and Vinod Vaikuntanathan. Bounded CCA2-secure encryption. In Kaoru Kurosawa, editor, *Advances in Cryptology - ASIACRYPT 2007, 13th International Conference on the Theory and Application of Cryptology and Information Security, Kuching, Malaysia, December 2-6, 2007, Proceedings*, volume 4833 of *Lecture Notes in Computer Science*, pages 502–518. Springer, 2007.
- [24] Eric Crockett, Christian Paquin, and Douglas Stebila. Prototyping post-quantum and hybrid key exchange and authentication in TLS and SSH. *IACR Cryptol. ePrint Arch.*, page 858, 2019.

- [25] Alexander W. Dent. A designer’s guide to kems. In Kenneth G. Paterson, editor, *Cryptography and Coding, 9th IMA International Conference, Cirencester, UK, December 16-18, 2003, Proceedings*, volume 2898 of *Lecture Notes in Computer Science*, pages 133–151. Springer, 2003.
- [26] Tim Dierks and Eric Rescorla. The transport layer security (TLS) protocol version 1.2. *RFC*, 5246:1–104, 2008.
- [27] Ronny Döring and Marc Geitz. Post-quantum cryptography in use: Empirical analysis of the TLS handshake performance. In *2022 IEEE/IFIP Network Operations and Management Symposium, NOMS 2022, Budapest, Hungary, April 25-29, 2022*, pages 1–5. IEEE, 2022.
- [28] Benjamin Dowling, Marc Fischlin, Felix Günther, and Douglas Stebila. A cryptographic analysis of the TLS 1.3 handshake protocol candidates. In Indrajit Ray, Ninghui Li, and Christopher Kruegel, editors, *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security, Denver, CO, USA, October 12-16, 2015*, pages 1197–1210. ACM, 2015.
- [29] Benjamin Dowling, Marc Fischlin, Felix Günther, and Douglas Stebila. A cryptographic analysis of the TLS 1.3 draft-10 full and pre-shared key handshake protocol. *IACR Cryptol. ePrint Arch.*, page 81, 2016.
- [30] Benjamin Dowling, Marc Fischlin, Felix Günther, and Douglas Stebila. A cryptographic analysis of the TLS 1.3 handshake protocol. *J. Cryptol.*, 34(4):37, 2021.
- [31] Marc Fischlin and Felix Günther. Multi-stage key exchange and the case of google’s QUIC protocol. In Gail-Joon Ahn, Moti Yung, and Ninghui Li, editors, *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security, Scottsdale, AZ, USA, November 3-7, 2014*, pages 1193–1204. ACM, 2014.
- [32] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael J. Wiener, editor, *Advances in Cryptology - CRYPTO ’99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, volume 1666 of *Lecture Notes in Computer Science*, pages 537–554. Springer, 1999.
- [33] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. *J. Cryptol.*, 26(1):80–101, 2013.

- [34] Xinwei Gao, Jintai Ding, Lin Li, Saraswathy RV, and Jiqiang Liu. Efficient implementation of password-based authenticated key exchange from RLWE and post-quantum TLS. *IACR Cryptol. ePrint Arch.*, page 1192, 2017.
- [35] Xinwei Gao, Jintai Ding, Lin Li, Saraswathy RV, and Jiqiang Liu. Efficient implementation of password-based authenticated key exchange from RLWE and post-quantum TLS. *Int. J. Netw. Secur.*, 20(5):923–930, 2018.
- [36] Xinwei Gao, Lin Li, Jintai Ding, Jiqiang Liu, R. V. Saraswathy, and Zhe Liu. Fast discretized gaussian sampling and post-quantum TLS ciphersuite. In Joseph K. Liu and Pierangela Samarati, editors, *Information Security Practice and Experience - 13th International Conference, ISPEC 2017, Melbourne, VIC, Australia, December 13-15, 2017, Proceedings*, volume 10701 of *Lecture Notes in Computer Science*, pages 551–565. Springer, 2017.
- [37] Carlos Rubio Garcia, Abraham Cano Aguilera, Juan Jose Vegas Olmos, Idelfonso Tafur Monroy, and Simon Rommel. Quantum-resistant TLS 1.3: A hybrid solution combining classical, quantum and post-quantum cryptography. In *28th IEEE International Workshop on Computer Aided Modeling and Design of Communication Links and Networks , CAMAD 2023, Edinburgh, United Kingdom, November 6-8, 2023*, pages 246–251. IEEE, 2023.
- [38] Alexandre Augusto Giron, João Pedro Adami do Nascimento, Ricardo Custódio, Lucas Pandolfo Perin, and Víctor Mateu. Post-quantum hybrid KEMTLS performance in simulated and real network environments. In Abdelrahman Aly and Mehdi Tibouchi, editors, *Progress in Cryptology - LATINCRYPT 2023 - 8th International Conference on Cryptology and Information Security in Latin America, LATINCRYPT 2023, Quito, Ecuador, October 3-6, 2023, Proceedings*, volume 14168 of *Lecture Notes in Computer Science*, pages 293–312. Springer, 2023.
- [39] Ruben Gonzalez and Thom Wiggers. KEMTLS vs. post-quantum TLS: performance on embedded systems. In Lejla Batina, Stjepan Picek, and Mainack Mondal, editors, *Security, Privacy, and Applied Cryptography Engineering - 12th International Conference, SPACE 2022, Jaipur, India, December 9-12, 2022, Proceedings*, volume 13783 of *Lecture Notes in Computer Science*, pages 99–117. Springer, 2022.
- [40] Lov K. Grover. A fast quantum mechanical algorithm for database search. In Gary L. Miller, editor, *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing, Philadelphia, Pennsylvania, USA, May 22-24, 1996*, pages 212–219. ACM, 1996.

- [41] Felix Günther, Simon Rastikian, Patrick Towa, and Thom Wiggers. KEMTLS with delayed forward identity protection in (almost) a single round trip. In Giuseppe Ateniese and Daniele Venturi, editors, *Applied Cryptography and Network Security - 20th International Conference, ACNS 2022, Rome, Italy, June 20-23, 2022, Proceedings*, volume 13269 of *Lecture Notes in Computer Science*, pages 253–272. Springer, 2022.
- [42] Yacoub Hanna, Diana Pineda, Maryna Veksler, Manish Paudel, Kemal Akkaya, Mila Anastasova, and Reza Azarderakhsh. Integrating post-quantum TLS into the control plane of 5g networks. In *IEEE International Performance, Computing, and Communications Conference, IPCCC 2024, Orlando, FL, USA, November 22-24, 2024*, pages 1–8. IEEE, 2024.
- [43] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the fujisaki-okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *Theory of Cryptography - 15th International Conference, TCC 2017, Baltimore, MD, USA, November 12-15, 2017, Proceedings, Part I*, volume 10677 of *Lecture Notes in Computer Science*, pages 341–371. Springer, 2017.
- [44] James Howe, Tobias Oder, Markus Krausz, and Tim Güneysu. Standard lattice-based key encapsulation on embedded devices. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(3):372–393, 2018.
- [45] Loïs Huguenin-Dumittan and Serge Vaudenay. On ind-qcca security in the ROM and its applications - CPA security is sufficient for TLS 1.3. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 - June 3, 2022, Proceedings, Part III*, volume 13277 of *Lecture Notes in Computer Science*, pages 613–642. Springer, 2022.
- [46] Panos Kampanakis and Will Childs-Klein. The impact of data-heavy, post-quantum TLS 1.3 on the time-to-last-byte of real-world connections. *IACR Cryptol. ePrint Arch.*, page 176, 2024.
- [47] Panos Kampanakis and Michael G. Kallitsis. Faster post-quantum TLS handshakes without intermediate CA certificates. In Shlomi Dolev, Jonathan Katz, and Amnon Meisels, editors, *Cyber Security, Cryptology, and Machine Learning - 6th International Symposium, CSCML 2022, Be’er Sheva, Israel, June 30 - July 1, 2022, Proceedings*, volume 13301 of *Lecture Notes in Computer Science*, pages 337–355. Springer, 2022.

- [48] Matthias J. Kannwischer, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. pqm4: Testing and benchmarking NIST PQC on ARM cortex-m4. *IACR Cryptol. ePrint Arch.*, page 844, 2019.
- [49] Brian Koziel, Reza Azarderakhsh, and Mehran Mozaffari Kermani. A high-performance and scalable hardware architecture for isogeny-based cryptography. *IEEE Trans. Computers*, 67(11):1594–1609, 2018.
- [50] Hugo Krawczyk. Cryptographic extraction and key derivation: The HKDF scheme. In Tal Rabin, editor, *Advances in Cryptology - CRYPTO 2010, 30th Annual Cryptology Conference, Santa Barbara, CA, USA, August 15-19, 2010. Proceedings*, volume 6223 of *Lecture Notes in Computer Science*, pages 631–648. Springer, 2010.
- [51] Hugo Krawczyk and Pasi Eronen. Hmac-based extract-and-expand key derivation function (HKDF). *RFC*, 5869:1–14, 2010.
- [52] Hugo Krawczyk and Hoeteck Wee. The OPTLS protocol and TLS 1.3. In *IEEE European Symposium on Security and Privacy, EuroS&P 2016, Saarbrücken, Germany, March 21-24, 2016*, pages 81–96. IEEE, 2016.
- [53] L LAMPORT. Constructing digital signatures from a one-way function. *Report SRI Intl. CSL 98*, 1979.
- [54] Dimitri Mankowski, Thom Wiggers, and Veelasha Moonsamy. TLS $\rightarrow$ post-quantum TLS: inspecting the TLS landscape for PQC adoption on android. In *IEEE European Symposium on Security and Privacy, EuroS&P 2023 - Workshops, Delft, Netherlands, July 3-7, 2023*, pages 526–538. IEEE, 2023.
- [55] Dimitri Mankowski, Thom Wiggers, and Veelasha Moonsamy. TLS $\hat{\rightarrow}$ post-quantum TLS: inspecting the TLS landscape for PQC adoption on android. *IACR Cryptol. ePrint Arch.*, page 734, 2023.
- [56] Dominik Marchsreiter and Johanna Sepúlveda. Hybrid post-quantum enhanced TLS 1.3 on embedded devices. In *25th Euromicro Conference on Digital System Design, DSD 2022, Maspalomas, Spain, August 31 - Sept. 2, 2022*, pages 905–912. IEEE, 2022.
- [57] Callum McLoughlin, Clémentine Gritti, and Juliet Samandari. Full post-quantum datagram TLS handshake in the internet of things. In Said El Hajji, Sihem Mesnager, and El Mamoun Souidi, editors, *Codes, Cryptology and Information Security - 4th International Conference, C2SI 2023, Rabat, Morocco, May 29-31, 2023, Proceedings*, volume 13874 of *Lecture Notes in Computer Science*, pages 57–76. Springer, 2023.

- [58] Simon Meier, Benedikt Schmidt, Cas Cremers, and David A. Basin. The TAMARIN prover for the symbolic analysis of security protocols. In Natasha Sharygina and Helmut Veith, editors, *Computer Aided Verification - 25th International Conference, CAV 2013, Saint Petersburg, Russia, July 13-19, 2013. Proceedings*, volume 8044 of *Lecture Notes in Computer Science*, pages 696–701. Springer, 2013.
- [59] National Institute of Standards and Technology. Security (evaluation criteria) — post-quantum cryptography standardization, 2017. Accessed: 2025-06-24.
- [60] Christian Paquin, Douglas Stebila, and Goutam Tamvada. Benchmarking post-quantum cryptography in TLS. *IACR Cryptol. ePrint Arch.*, page 1447, 2019.
- [61] Christian Paquin, Douglas Stebila, and Goutam Tamvada. Benchmarking post-quantum cryptography in TLS. In Jintai Ding and Jean-Pierre Tillich, editors, *Post-Quantum Cryptography - 11th International Conference, PQCrypto 2020, Paris, France, April 15-17, 2020, Proceedings*, volume 12100 of *Lecture Notes in Computer Science*, pages 72–91. Springer, 2020.
- [62] Sebastian Paul, Yulia Kuzovkova, Norman Lahr, and Ruben Niederhagen. Mixed certificate chains for the transition to post-quantum authentication in TLS 1.3. *IACR Cryptol. ePrint Arch.*, page 1447, 2021.
- [63] Sebastian Paul, Yulia Kuzovkova, Norman Lahr, and Ruben Niederhagen. Mixed certificate chains for the transition to post-quantum authentication in TLS 1.3. In Yuji Suga, Kouichi Sakurai, Xuhua Ding, and Kazue Sako, editors, *ASIA CCS '22: ACM Asia Conference on Computer and Communications Security, Nagasaki, Japan, 30 May 2022 - 3 June 2022*, pages 727–740. ACM, 2022.
- [64] Sebastian Paul, Felix Schick, and Jan Seedorf. Tpm-based post-quantum cryptography: A case study on quantum-resistant and mutually authenticated TLS for iot environments. In Delphine Reinhardt and Tilo Müller, editors, *ARES 2021: The 16th International Conference on Availability, Reliability and Security, Vienna, Austria, August 17-20, 2021*, pages 3:1–3:10. ACM, 2021.
- [65] Eric Rescorla. The transport layer security (TLS) protocol version 1.3. *RFC*, 8446:1–160, 2018.
- [66] Kadir Sabanci and Mumin Cebe. Exploring post-quantum cryptographic schemes for TLS in 5g nb-iot: Feasibility and recommendations. In Paul A. Pavlou, Vishal Midha, Animesh Animesh, Traci A. Carte, Alexandre R. Graeml, and Alanah Mitchell,

editors, *29th Americas Conference on Information Systems, AMCIS 2023, Panama City, Panama, August 10-12, 2023*. Association for Information Systems, 2023.

- [67] Kadir Sabanci and Mumin Cebe. Exploring post-quantum cryptographic schemes for TLS in 5g nb-iot: Feasibility and recommendations. *CoRR*, abs/2309.03338, 2023.
- [68] Maximilian Schöffel, Frederik Lauer, Carl Christian Rheinländer, and Norbert Wehn. On the energy costs of post-quantum kems in tls-based low-power secure iot. In *IoTDI '21: International Conference on Internet-of-Things Design and Implementation, Virtual Event / Charlottesville, VA, USA, May 18-21, 2021*, pages 158–168. ACM, 2021.
- [69] Peter Schwabe, Douglas Stebila, and Thom Wiggers. Post-quantum TLS without handshake signatures. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *CCS '20: 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, USA, November 9-13, 2020*, pages 1461–1480. ACM, 2020.
- [70] Peter Schwabe, Douglas Stebila, and Thom Wiggers. More efficient post-quantum KEMTLS with pre-distributed public keys. In Elisa Bertino, Haya Schulmann, and Michael Waidner, editors, *Computer Security - ESORICS 2021 - 26th European Symposium on Research in Computer Security, Darmstadt, Germany, October 4-8, 2021, Proceedings, Part I*, volume 12972 of *Lecture Notes in Computer Science*, pages 3–22. Springer, 2021.
- [71] Michael Scott. On TLS for the internet of things, in a post quantum world. *IACR Cryptol. ePrint Arch.*, page 95, 2023.
- [72] Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *35th Annual Symposium on Foundations of Computer Science, Santa Fe, New Mexico, USA, 20-22 November 1994*, pages 124–134. IEEE Computer Society, 1994.
- [73] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM J. Comput.*, 26(5):1484–1509, 1997.
- [74] Dimitrios Sikeridis, Sean Huntley, David Ott, and Michael Devetsikiotis. Intermediate certificate suppression in post-quantum TLS: an approximate membership querying approach. In Giuseppe Bianchi and Alessandro Mei, editors, *Proceedings of the 18th International Conference on emerging Networking EXperiments and Technologies, CoNEXT 2022, Roma, Italy, December 6-9, 2022*, pages 35–42. ACM, 2022.

- [75] Dimitrios Sikeridis, Sean Huntley, David Ott, and Michael Devetsikiotis. Intermediate certificate suppression in post-quantum TLS: an approximate membership querying approach. *IACR Cryptol. ePrint Arch.*, page 1556, 2022.
- [76] Dimitrios Sikeridis, Panos Kampanakis, and Michael Devetsikiotis. Assessing the overhead of post-quantum cryptography in TLS 1.3 and SSH. In Dongsu Han and Anja Feldmann, editors, *CoNEXT '20: The 16th International Conference on emerging Networking EXperiments and Technologies, Barcelona, Spain, December, 2020*, pages 149–156. ACM, 2020.
- [77] Dimitrios Sikeridis, Panos Kampanakis, and Michael Devetsikiotis. Post-quantum authentication in TLS 1.3: A performance study. In *27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23-26, 2020*. The Internet Society, 2020.
- [78] Dimitrios Sikeridis, Panos Kampanakis, and Michael Devetsikiotis. Post-quantum authentication in TLS 1.3: A performance study. *IACR Cryptol. ePrint Arch.*, page 71, 2020.
- [79] Markus Sosnowski, Florian Wiedner, Eric Hauser, Lion Steger, Dimitrios Schoini-anakis, Sebastian Gallenmüller, and Georg Carle. The performance of post-quantum TLS 1.3. In Dario Rossi, Stefano Secci, Olivier Bonaventure, and Lili Qiu, editors, *Companion of the 19th International Conference on emerging Networking EXperiments and Technologies, CoNEXT 2023 (Short Papers), Paris, France, December 5-8, 2023*, pages 19–27. ACM, 2023.
- [80] George Tasopoulos, Charis Dimopoulos, Apostolos P. Fournaris, Raymond K. Zhao, Amin Sakzad, and Ron Steinfeld. Energy consumption evaluation of post-quantum TLS 1.3 for resource-constrained embedded devices. In Andrea Bartolini, Kristian F. D. Rietveld, Catherine D. Schuman, and Jose Moreira, editors, *Proceedings of the 20th ACM International Conference on Computing Frontiers, CF 2023, Bologna, Italy, May 9-11, 2023*, pages 366–374. ACM, 2023.
- [81] George Tasopoulos, Charis Dimopoulos, Apostolos P. Fournaris, Raymond K. Zhao, Amin Sakzad, and Ron Steinfeld. Energy consumption evaluation of post-quantum TLS 1.3 for resource-constrained embedded devices. *IACR Cryptol. ePrint Arch.*, page 506, 2023.
- [82] George Tasopoulos, Jinhui Li, Apostolos P. Fournaris, Raymond K. Zhao, Amin Sakzad, and Ron Steinfeld. Performance evaluation of post-quantum TLS 1.3 on embedded systems. *IACR Cryptol. ePrint Arch.*, page 1553, 2021.

- [83] George Tasopoulos, Jinhui Li, Apostolos P. Fournaris, Raymond K. Zhao, Amin Sakzad, and Ron Steinfeld. Performance evaluation of post-quantum TLS 1.3 on resource-constrained embedded systems. Cryptology ePrint Archive, Paper 2021/1553, 2021.
- [84] George Tasopoulos, Jinhui Li, Apostolos P. Fournaris, Raymond K. Zhao, Amin Sakzad, and Ron Steinfeld. Performance evaluation of post-quantum TLS 1.3 on resource-constrained embedded systems. In Chunhua Su, Dimitris Gritzalis, and Vincenzo Piuri, editors, *Information Security Practice and Experience - 17th International Conference, ISPEC 2022, Taipei, Taiwan, November 23-25, 2022, Proceedings*, volume 13620 of *Lecture Notes in Computer Science*, pages 432–451. Springer, 2022.
- [85] Duong Dinh Tran, Canh Minh Do, Santiago Escobar, and Kazuhiro Ogata. Hybrid post-quantum TLS formal specification in maude-npa - toward its security analysis. In Sedat Akleyek, Santiago Escobar, Kazuhiro Ogata, and Ayoub Otmani, editors, *Proceedings of the International Workshop on Formal Analysis and Verification of Post-Quantum Cryptographic Protocols co-located with the 23rd International Conference on Formal Engineering Methods (ICFEM 2022), Madrid, Spain, October 24, 2022*, volume 3280 of *CEUR Workshop Proceedings*, pages 50–64. CEUR-WS.org, 2022.
- [86] Bas Westerbaan. The state of the post-quantum internet, Mar 2024.
- [87] Jieyu Zheng, Haoliang Zhu, Yifan Dong, Zhenyu Song, Zhenhao Zhang, Yafang Yang, and Yunlei Zhao. Faster post-quantum TLS 1.3 based on ML-KEM: implementation and assessment. In Joaquín García-Alfaro, Rafal Kozik, Michal Choras, and Sokratis K. Katsikas, editors, *Computer Security - ESORICS 2024 - 29th European Symposium on Research in Computer Security, Bydgoszcz, Poland, September 16-20, 2024, Proceedings, Part II*, volume 14983 of *Lecture Notes in Computer Science*, pages 123–143. Springer, 2024.
- [88] Biming Zhou, Haodong Jiang, and Yunlei Zhao. CPA-secure KEMs are also sufficient for post-quantum TLS 1.3. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part III*, volume 15486 of *Lecture Notes in Computer Science*, pages 433–464. Springer, 2024.

APPENDICES